

FRONTEND INTERVIEW REFERENCE

React JS Interview Prep Guide

70 Essential Topics for 0–3 Years React Developers

A consolidated, exam-style reference covering React fundamentals, JSX, components, hooks mastery, state management (Context + Redux Toolkit), routing, performance optimisation, testing, TypeScript, Vite, Next.js and server-side rendering, React internals, common interview questions, machine coding challenges, frontend system design, project ideas, and HR questions you are likely to face — each with definitions, code examples, and trade-offs you can recall in 60 seconds.

- JSX
- Hooks
- useState
- useEffect
- useContext
- useMemo
- useCallback
- Redux Toolkit
- React Router
- TanStack Query
- TypeScript
- Vite
- Next.js
- SSR
- Testing

EXPERIENCE
0–3 Years

TOPICS
70 Sections

TARGET
React 19 / Vite

Table of Contents

Foreword	9
1. Introduction to React	10
1.1 Key Facts	10
1.2 React Ecosystem at a Glance	10
1.3 Create a React App	10
1.4 Hello World	11
2. React Features	11
2.1 Core Features	11
2.2 Declarative vs Imperative	11
2.3 Unidirectional Data Flow	12
3. JSX	12
3.1 Basic JSX	12
3.2 JSX Rules	12
3.3 Conditional Rendering in JSX	13
3.4 Lists in JSX	13
4. Components	13
4.1 Component Types	13
4.2 Function Component	14
4.3 Component Composition	14
4.4 Component Naming & File Organisation	14
5. Props	15
5.1 Passing and Receiving Props	15
5.2 Props Are Read-Only	15
5.3 Default Props	15
5.4 Spread Props	15
5.5 Children Prop	16
5.6 Prop Drilling Problem	16
6. State	16
6.1 useState Hook	16
6.2 State Updates Are Asynchronous	16
6.3 useState Patterns	17

6.4 State vs Derived Value	17
7. Event Handling	18
7.1 Basic Event Handlers	18
7.2 SyntheticEvent	18
7.3 Passing Arguments to Handlers	18
8. Conditional Rendering	19
8.1 Patterns	19
8.2 Element Variables	19
8.3 Show/Hide with CSS	20
9. Lists and Keys	20
9.1 Basic List	20
9.2 Why Keys Matter	20
9.3 Index Key — When It's OK	21
9.4 Anti-Patterns	21
10. Forms	21
10.1 Controlled Form	21
10.2 Validation Patterns	22
10.3 For Complex Forms — Use a Library	22
11. Lifting State Up	23
11.1 Example: Synchronised Inputs	23
11.2 When to Lift State	23
12. Controlled vs Uncontrolled Components	23
12.1 Comparison	24
12.2 Controlled Example	24
12.3 Uncontrolled Example	24
12.4 When to Use Each	24
13. React Lifecycle	25
13.1 Lifecycle Phases	25
13.2 Class Lifecycle Methods	25
13.3 Function Equivalent (useEffect)	25
14. Functional Components vs Class Components	26
14.1 Comparison	26
14.2 Side-by-Side: Counter	26
14.3 Why Function Components Won	27

14.4 When Classes Are Still Used	27
15. Hooks	28
15.1 Built-in Hooks	28
15.2 Rules of Hooks	28
15.3 Why Hooks?	29
16. useState	29
16.1 Basic Usage	29
16.2 Functional Updates	29
16.3 Lazy Initialisation	29
16.4 Object State — Always Replace, Never Mutate	30
16.5 When to Use useReducer Instead	30
17. useEffect	30
17.1 Anatomy	30
17.2 Common Patterns	31
17.3 Cleanup Function	31
17.4 Common Pitfalls	31
17.5 useEffect vs useLayoutEffect	32
18. useContext	32
18.1 Three Steps	32
18.2 When to Use Context	32
18.3 Context Performance	33
18.4 Custom Hook for Context	33
19. useRef	33
19.1 DOM Access	33
19.2 Mutable Value Container	34
19.3 useRef vs useState	34
19.4 Storing "Previous" Value	35
20. useMemo	35
20.1 Basic Pattern	35
20.2 When to Use	35
20.3 When NOT to Use	36
21. useCallback	36
21.1 Basic Pattern	36
21.2 When to Use	36

21.3 When NOT to Use	36
21.4 useCallback vs useMemo	37
22. useReducer	37
22.1 Counter with useReducer	37
22.2 useState vs useReducer	38
23. Custom Hooks	38
23.1 Anatomy	38
23.2 Common Custom Hooks	38
23.3 Rules	39
24. React Router	39
24.1 Basic Setup	40
24.2 URL Parameters	40
24.3 Navigation	40
24.4 Lazy-Loaded Routes	41
25. State Management	41
25.1 State Categories	41
25.2 Choose the Right Tool	42
26. Redux Toolkit	42
26.1 Setup	42
26.2 Using in Components	43
26.3 Async with createAsyncThunk	43
27. Context API	44
27.1 Provider Pattern	44
27.2 Splitting Contexts	44
27.3 Context vs Redux	44
28. Performance Optimization	45
28.1 Tools	45
28.2 Optimisation Techniques	45
28.3 Common Pitfalls	46
29. React.memo	46
29.1 Basic Usage	46
29.2 Custom Comparator	46
29.3 When to Use	47
29.4 When NOT to Use	47

30. Lazy Loading	47
30.1 React.lazy	47
30.2 Lazy Loading Routes	48
31. Suspense	48
31.1 Basic Suspense	48
31.2 Nested Suspense	48
31.3 Suspense for Data Fetching (React 19 / TanStack Query)	48
32. Error Boundaries	49
32.1 Class-Based Error Boundary	49
32.2 react-error-boundary (Recommended)	49
32.3 What Error Boundaries Do NOT Catch	50
33. Portals, HOCs, Render Props, Compound Components	50
33.1 Portals	50
33.2 Higher-Order Components (HOCs)	51
33.3 Render Props	51
33.4 Compound Components	52
34. Styling in React	53
34.1 Options at a Glance	53
34.2 Inline Styles	53
35. CSS Modules	53
35.1 Usage	54
35.2 Pros & Cons	54
36. Styled Components	54
36.1 Basic Usage	54
36.2 Pros & Cons	55
37. Tailwind CSS	55
37.1 Basic Usage	55
37.2 Pros & Cons	55
38. API Calls	56
38.1 Fetching in useEffect	56
38.2 Why You Should Use TanStack Query Instead	56
39. Axios vs Fetch	57
39.1 Comparison	57
39.2 Fetch Example	57

39.3 Axios Example	57
40. Authentication	58
40.1 Token Storage Trade-offs	58
40.2 Auth Context Pattern	58
41. JWT Authentication	59
41.1 JWT Structure	59
41.2 Client-side JWT Flow	59
42. Protected Routes	59
42.1 Pattern	60
42.2 Role-Based Protection	60
43. React Query / TanStack Query	61
43.1 Setup	61
43.2 Basic Query	61
43.3 Mutations	61
44. Form Libraries	62
44.1 React Hook Form (Recommended)	62
44.2 Why React Hook Form?	63
45. Testing in React	63
45.1 Test Types	63
45.2 Guiding Principles	64
46. Jest	64
46.1 Basic Test	64
46.2 Mocking	64
46.3 Vitest (Modern Alternative)	65
47. React Testing Library	65
47.1 Basic Component Test	65
47.2 Query Priority (Use in This Order)	65
48. TypeScript with React	66
48.1 Typing Props	66
48.2 Typing Hooks	66
48.3 Typing Events	67
48.4 Utility Types	67
49. Folder Structure	68
49.1 Feature-Based (Recommended)	68

49.2 Why Feature-Based?	68
50. Build Tools	69
50.1 Vite (Recommended)	69
50.2 Next.js	69
51. Vite vs CRA	69
51.1 Comparison	69
51.2 Recommendation	70
52. SSR vs CSR	70
52.1 Comparison	70
52.2 Other Rendering Modes	70
53. Next.js Basics	71
53.1 App Router Structure	71
53.2 Server vs Client Components	71
53.3 Data Fetching	71
54. Virtual DOM	72
54.1 Why VDOM?	72
54.2 VDOM Tree Example	72
55. Reconciliation & Diffing	72
55.1 Diffing Heuristics	73
55.2 Keys in Reconciliation	73
56. React Fiber	73
56.1 Why Fiber?	73
56.2 Fiber Tree	74
58. Machine Coding Questions	74
58.1 Build a Counter	74
58.2 Build a Todo List	74
58.3 Build an Autocomplete / Typeahead	75
58.4 Common Machine Coding Tasks	76
59. Coding Challenges	76
59.1 Implement Debounce	77
59.2 Implement Throttle	77
59.3 Implement Promise.all	77
59.4 Implement usePrevious Hook	77
59.5 Implement useInterval Hook	78

60. System Design for Frontend	78
60.1 Six-Step Framework	78
60.2 Example: Design an Autocomplete Search	78
60.3 Performance Checklist	79
61. Best Practices	79
61.1 Component Design	79
61.2 State	79
61.3 Effects	80
61.4 Performance	80
61.5 Accessibility	80
61.6 Testing	80
62. React Project Ideas	80
62.1 Beginner (1-2 days each)	81
62.2 Intermediate (1-2 weeks each)	81
62.3 Advanced (1-2 months)	81
62.4 Tips for Portfolio Projects	81
63. HR Interview Questions	82
63.1 STAR Framework	82
63.4 Questions to Ask the Interviewer	82
63.5 Final Tips	82

Foreword

This guide consolidates 70 essential topics that a React JS developer with zero to three years of experience is expected to know in interviews and day-to-day frontend work. Each topic is structured as a concise definition, the core concepts you must internalise, runnable code examples, common pitfalls, and one or two interview-style questions with model answers. The aim is to give you a single revision document that you can read end-to-end in a few hours and revisit before any interview round.

The material reflects the realistic expectations of junior-to-mid React roles: solid JSX and component fundamentals, hooks mastery, state management (Context + Redux Toolkit), routing, performance optimisation, testing, TypeScript, modern tooling (Vite), and an awareness of Next.js and server-side rendering. The closing sections focus on common interview questions, machine coding challenges, frontend system design, best practices, project ideas, and HR questions you are likely to face.

How to use this guide: Read each topic in order; do not skip the code examples even if the concept feels familiar. Most interview rejections happen because a candidate can describe a hook but cannot write the syntax from memory. After finishing a topic, attempt to write the example without looking, then compare with the printed version. Pay extra attention to hooks (15-23), state management (24-26), performance (27-30), and system design (62) — these are the highest-frequency mid-level interview topics.

1. Introduction to React

React is an open-source JavaScript library for building user interfaces, originally developed at Facebook (now Meta) and released in 2013. It focuses on the view layer of an application, leaving routing, state management, and data fetching to complementary libraries. React popularised the component-based architecture and the virtual DOM, two ideas that reshaped how the industry thinks about UI development.

1.1 Key Facts

- **Created by** — Jordan Walke at Facebook (2013 open-sourced).
- **Maintained by** — Meta and the React community.
- **Type** — Library, not a full framework (no router, no state management built in).
- **Rendering** — Client-side by default; server-side via Next.js, Remix.
- **Paradigm** — Declarative, component-based, unidirectional data flow.
- **License** — MIT.
- **Latest stable** — React 19 (2024+). React 18 introduced concurrent features.

1.2 React Ecosystem at a Glance

Concern	Popular Options
Routing	React Router, Next.js App Router, TanStack Router
State management	Context API, Redux Toolkit, Zustand, Jotai, Recoil
Data fetching	TanStack Query (React Query), SWR, Apollo Client (GraphQL)
Build tools	Vite, Next.js, Remix, Astro
Styling	CSS Modules, Tailwind CSS, styled-components, Emotion, vanilla-extract
Forms	React Hook Form, Formik, TanStack Form
Testing	Jest, Vitest, React Testing Library, Playwright, Cypress
Type checking	TypeScript (recommended), PropTypes (legacy)

1.3 Create a React App

```
// Modern: Vite (recommended for new projects)
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev

// Or: Next.js (full-stack React framework)
npx create-next-app@latest my-app

// Legacy: Create React App (CRA) - deprecated in 2023, avoid for new projects
// npx create-react-app my-app // [X] deprecated
```

1.4 Hello World

```
import { createRoot } from 'react-dom/client';

function App() {
  return <h1>Hello, React!</h1>;
}

const root = createRoot(document.getElementById('root')!);
root.render(<App />);
```

2. React Features

React's defining features — the virtual DOM, JSX, components, one-way data binding, and hooks — together create a productive, predictable UI development model. Understanding each feature's purpose (not just the syntax) is what separates junior from mid-level candidates.

2.1 Core Features

Feature	What It Does	Benefit
Virtual DOM	In-memory representation of the DOM; React diffs and updates only changes	Performance
JSX	HTML-like syntax compiled to React.createElement calls	Readability
Components	Reusable, composable UI building blocks	Reuse, structure
One-way data binding	Data flows parent → child via props	Predictability
State & props	Mutable internal state vs immutable inputs	Reactivity
Hooks	useState, useEffect, etc. — state and side effects in function components	Function components
Concurrent features	React 18: time slicing, transitions, Suspense for data	Responsiveness
Server Components	React 18+: components that render only on the server	Bundle size

2.2 Declarative vs Imperative

React is declarative: you describe what the UI should look like for a given state, and React figures out how to update the DOM to match. Imperative code (jQuery style) tells the browser step-by-step what to do (find element, set text, add class). Declarative code is shorter, easier to reason about, and less bug-prone because React handles the diffing.

```
// Imperative (jQuery-style)
document.getElementById('count').textContent = count;
document.getElementById('count').classList.add('highlight');

// Declarative (React)
function Counter({ count }) {
```

```
return <h1 className={count > 10 ? 'highlight' : ''}>{count}</h1>;
}
```

2.3 Unidirectional Data Flow

In React, data flows from parent to child via props. A child cannot modify its parent's state directly — it can only call a callback passed down via props. This one-way binding makes state changes traceable: to find what changed a piece of state, look for who calls the setter. Two-way binding (Vue, Angular) is convenient but harder to debug at scale.

3. JSX

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside your JavaScript. It is not a template language — it compiles to plain `React.createElement()` calls. JSX makes component code readable and is the default way to write React components.

3.1 Basic JSX

```
function Welcome({ name }) {
  return (
    <div className="greeting">
      <h1>Hello, {name}!</h1>
      <p>Today is {new Date().toLocaleDateString()}</p>
    </div>
  );
}

// Compiles to:
function Welcome({ name }) {
  return React.createElement('div', { className: 'greeting' },
    React.createElement('h1', null, 'Hello, ', name, '!'),
    React.createElement('p', null, 'Today is ', new Date().toLocaleDateString(), '.')
  );
}
```

3.2 JSX Rules

- **Single root element** — A component must return one parent (or a Fragment (`<>...</>`)). Multiple siblings without a wrapper = syntax error.
- **className, not class** — `class` is a reserved word in JS; use `className`. Same for `htmlFor` (not `for`).
- **Self-closing tags** — Void elements must close: `<br />`, `<img />`, `<input />`. Or use `<input></input>` style.
- **camelCase attributes** — `tabIndex` not `tabindex`, `onClick` not `onclick`, `maxLength` not `maxlength`.
- **style is an object** — `style={{ color: 'red', fontSize: 14 }}` — camelCase keys, string or number values.
- **Expressions in {}** — `{condition ? 'a' : 'b'}` — any JS expression goes inside braces. Statements (`if`, `for`) do NOT.
- **Booleans/null render nothing** — `{true && }` renders nothing. `{false}` alone also renders nothing. `{0}` renders "0" — gotcha.

3.3 Conditional Rendering in JSX

```
function Greeting({ isLoggedIn, user }) {
  return (
    <div>
      {/* Ternary */}
      {isLoggedIn ? <h1>Welcome back, {user.name}</h1> : <h1>Please sign in</h1>}

      {/* Logical && (careful with numbers!) */}
      {user.unreadCount > 0 && <Badge count={user.unreadCount} />}

      {/* IIFE for if-else */}
      {(() => {
        if (user.role === 'admin') return <AdminPanel />;
        if (user.role === 'user') return <UserPanel />;
        return <GuestPanel />;
      })()}
    </div>
  );
}
```

3.4 Lists in JSX

```
function ProductList({ products }) {
  return (
    <ul>
      {products.map(p => (
        <li key={p.id}>
          {p.name} - ${p.price}
        </li>
      ))}
    </ul>
  );
}

// ! NEVER use array index as key if list can reorder or items can be inserted.
// Use a stable unique ID from your data (p.id).
```

4. Components

Components are the building blocks of a React application. A component is a JavaScript function (or class) that returns JSX describing what should appear on the screen. Components are composed to form the UI tree.

4.1 Component Types

Type	Definition	Use Case
Function component	JS function returning JSX	Modern default — use this
Class component	ES6 class extending <code>React.Component</code> with <code>render()</code>	Legacy; needed for error boundaries before React 19
Pure component	Class with shallow prop/state comparison	<code>React.PureComponent</code> — auto-memoised
Memoised function	Function wrapped in <code>React.memo()</code>	Skip re-renders if props unchanged

Type	Definition	Use Case
Higher-Order Component	Function taking a component, returning a component	Cross-cutting concerns

4.2 Function Component

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
}

function Button({ label, onClick, variant = 'primary' }: ButtonProps) {
  return (
    <button
      className={`btn btn-${variant}`}
      onClick={onClick}
    >
      {label}
    </button>
  );
}

export default Button;
```

4.3 Component Composition

Compose small, single-purpose components into larger ones via the children prop or explicit props. Composition is preferred over inheritance in React — the team's official guidance is "use composition, not inheritance".

```
function Card({ title, children, footer }) {
  return (
    <div className="card">
      <header><h2>{title}</h2></header>
      <main>{children}</main>
      {footer && <footer>{footer}</footer>}
    </div>
  );
}

// Usage: pass any JSX as children
<Card title="Profile" footer={<SaveButton />}>
  <Avatar url={user.avatar} />
  <UserInfo user={user} />
</Card>
```

4.4 Component Naming & File Organisation

- **PascalCase names** — Button, ProductList, UserProfile — distinguishes from HTML tags (button, div).
- **One component per file** — Easier to test, find, and refactor.
- **Co-locate tests, styles** — Button.tsx, Button.test.tsx, Button.module.css.
- **Default export for components** — Default for the main component; named exports for variants/sub-components.

5. Props

Props (short for "properties") are the inputs to a component. They are passed from a parent and are read-only inside the child — a component must never modify its own props. Props make components reusable and composable.

5.1 Passing and Receiving Props

```
// Parent
function App() {
  return <UserCard name="Alice" age={30} isActive={true} tags={['admin', 'vip']} />;
}

// Child – destructure props
function UserCard({ name, age, isActive, tags = [] }) {
  return (
    <div>
      <h1>{name} ({age})</h1>
      {isActive} && <span className="badge">Active</span>
      <ul>{tags.map(t => <li key={t}>{t}</li>)}</ul>
    </div>
  );
}
```

5.2 Props Are Read-Only

- **Pure function rule** — A component must behave like a pure function of its props — same props → same output. Never mutate props inside a component.
- **State for changes** — If a value needs to change, it belongs in state, not props.
- **Pass callbacks up** — To communicate child → parent, pass a callback prop the child calls; the parent updates its state and re-renders.

5.3 Default Props

```
// Destructuring with default (preferred)
function Button({ variant = 'primary', size = 'md' }: ButtonProps) {
  // ...
}

// Legacy: Button.defaultProps (deprecated in React 18.3 for function components)
// Use destructuring defaults instead.
```

5.4 Spread Props

```
// Spread: forward all props to a child element
function StyledButton({ variant, ...rest }) {
  // variant is extracted; rest contains onClick, disabled, etc.
  return <button className={`btn btn-${variant}`} {...rest} />;
}

// Usage
<StyledButton variant="primary" onClick={...} disabled={false}>Save</StyledButton>
```


5.5 Children Prop

```
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;
  return (
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>
  );
}

<Modal isOpen={open} onClose={() => setOpen(false)}>
  <h2>Are you sure?</h2>
  <button>Confirm</button>
</Modal>
```

5.6 Prop Drilling Problem

When a piece of state must pass through many intermediate components to reach a deep child, you get "prop drilling" — every middle component receives and forwards props it does not use. Solution: Context API (for rarely-changing app-wide state) or a state library (Redux Toolkit, Zustand) for frequently-updating state.

6. State

State is data that a component owns and that can change over time. When state changes, React re-renders the component (and its descendants) to reflect the new state. State is what makes React apps interactive.

6.1 useState Hook

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // initial value 0

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

6.2 State Updates Are Asynchronous

- **Batched** — React batches multiple `setState` calls in event handlers into one re-render. In React 18, batching applies to all updates (events, timeouts, promises).
- **Use functional form for previous value** — `setCount(c => c + 1)` — guaranteed correct even if called multiple times in a row. `setCount(count + 1)` called twice in the same tick increments once (closure captures stale count).

- **New object identity = re-render** — `setState` with the same reference (e.g., `setArr(arr)`) does not re-render. Always create a new array/object.

```
// [X] WRONG – uses stale `count` (closure)
function increment() {
  setCount(count + 1); // 0 + 1 = 1
  setCount(count + 1); // 0 + 1 = 1 (closure has old count)
}

// OK RIGHT – functional update
function increment() {
  setCount(c => c + 1); // 0 + 1 = 1
  setCount(c => c + 1); // 1 + 1 = 2
}

// [X] WRONG – mutates array (no re-render)
function addItem(item) {
  items.push(item);
  setItems(items); // same reference → no re-render
}

// OK RIGHT – new array
function addItem(item) {
  setItems(prev => [...prev, item]);
}
```

6.3 useState Patterns

- **Lazy initial state** — `useState(() => expensiveInit())` — the function runs only on first render. Use for expensive computations like reading `localStorage`.
- **Multiple states** — Prefer multiple `useState` calls over one big object. React optimises each independently. Big state objects force full re-creation on every update.
- **Derived state** — Do not store values you can compute: `const isEven = count % 2 === 0` — compute during render, not in state.
- **Controlled inputs** — `<input value={text} onChange={e => setText(e.target.value)} />` — React owns the value.
- **Lift state up** — When two siblings need the same state, move it to their common parent.

6.4 State vs Derived Value

```
// [X] Anti-pattern: storing derived value in state
function Cart({ items }) {
  const [total, setTotal] = useState(0);
  useEffect(() => {
    setTotal(items.reduce((s, i) => s + i.price, 0));
  }, [items]);
  // ...
}

// OK Right: derive during render
function Cart({ items }) {
  const total = items.reduce((s, i) => s + i.price, 0);
  // ...
}
```

7. Event Handling

React events use camelCase (onClick, not onclick) and accept a function reference rather than a string. React attaches a single delegate listener at the root and uses SyntheticEvent — a wrapper around the native event that normalises behaviour across browsers.

7.1 Basic Event Handlers

```
function Form() {
  const [email, setEmail] = useState('');
  const [agree, setAgree] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault(); // stop form reload
    console.log('Submit:', email, agree);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
      />
      <label>
        <input
          type="checkbox"
          checked={agree}
          onChange={e => setAgree(e.target.checked)}
        />
        I agree to terms
      </label>
      <button type="submit" disabled={!agree}>Submit</button>
    </form>
  );
}
```

7.2 SyntheticEvent

- **Cross-browser wrapper** — Normalises event properties (target, key, keyCode, etc.) across browsers.
- **Pool (pre-React 17)** — Events were pooled and recycled — accessing event.target in an async callback needed event.persist(). React 17+ removed pooling.
- **Native event** — event.nativeEvent gives the raw DOM event.
- **Default behaviour** — Call event.preventDefault() to stop the default (form submit, link click). event.stopPropagation() stops bubbling.

7.3 Passing Arguments to Handlers

```
// [X] WRONG – invokes immediately (not on click)
<button onClick={handleDelete(id)}>Delete</button>

// OK Option 1: arrow wrapper
<button onClick={() => handleDelete(id)}>Delete</button>

// OK Option 2: bind (creates new function each render too)
```

```

<button onClick={handleDelete.bind(null, id)}>Delete</button>

// OK Option 3: data attribute + read from event
<button data-id={id} onClick={handleDelete}>Delete</button>
function handleDelete(e) {
  const id = e.currentTarget.dataset.id;
}

// Note: arrow wrappers create new function each render.
// For perf-critical lists, use useCallback + the data-attribute pattern.

```

8. Conditional Rendering

React lets you conditionally render UI based on state, props, or any JavaScript expression. There are four common patterns: ternary, logical &&, IIFE, and extracting to a helper function. Each has trade-offs.

8.1 Patterns

```

function Dashboard({ user, isLoading, error }) {
  // 1. Ternary – either/or
  if (isLoading) return <Spinner />;
  if (error) return <ErrorMessage error={error} />;
  if (!user) return <LoginPrompt />;

  // 2. Inline ternary
  return (
    <div>
      {user.role === 'admin' ? <AdminPanel /> : <UserPanel />}
    </div>
  );
}

function Notification({ count }) {
  // 3. Logical && – show or nothing
  return (
    <div>
      {count > 0 && <Badge count={count} />}
    </div>
  );
}

// ! GOTCHA: {count && <Badge />} renders "0" when count is 0!
// Because 0 is a falsy value React will render. Fix:
// {count > 0 && <Badge />}
// {Boolean(count) && <Badge />}
// {!!count && <Badge />}

```

8.2 Element Variables

```

function Message({ type, text }) {
  let icon;
  if (type === 'success') icon = <CheckIcon />;
  else if (type === 'error') icon = <ErrorIcon />;
  else if (type === 'warning') icon = <WarningIcon />;
  else icon = <InfoIcon />;

  return (
    <div className={`msg msg-${type}`}>

```

```

    {icon}
    <span>{text}</span>
  </div>
);
}

```

8.3 Show/Hide with CSS

Sometimes you want to keep the element in the DOM but hide it visually (e.g., for animation or to preserve state). Use CSS instead of conditional rendering.

```

// Hide with CSS – keeps DOM and state
<div style={{ display: isVisible ? 'block' : 'none' }}>
  <ExpensiveComponent />
</div>

// VS unmount – state is lost
{isVisible && <ExpensiveComponent />}

// Use unmount when: state should reset, or component is expensive to keep.

```

9. Lists and Keys

Rendering lists of items is one of the most common React tasks. The pattern is `.map()` over an array, returning JSX for each item. Every list item must have a stable, unique `key` prop — React uses it to identify which items change across renders.

9.1 Basic List

```

function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id} className={todo.done ? 'done' : ''}>
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => toggleTodo(todo.id)}
          />
          {todo.text}
        </li>
      ))}
    </ul>
  );
}

```

9.2 Why Keys Matter

- **Identity across renders** — React matches elements by key between previous and next render. Without stable keys, React must assume the entire list changed.
- **Performance** — With stable keys, React reuses existing DOM nodes and updates only changed attributes — much faster than recreating the DOM.

- **State preservation** — If a list item contains stateful inputs (e.g., a text field), the right key ensures that state stays with the right item when the list reorders.
- **Avoid index as key** — If items can be inserted/deleted/reordered, index keys cause bugs: state attached to wrong item, animations jumpy, uncontrolled inputs misbehave.

9.3 Index Key — When It's OK

- **OK if list is static** — Never reordered, no insertions/deletions in the middle, items have no internal state.
- **OK if list is filtered only at end** — Items only appended or removed from the end; never reordered.
- **Never OK if items can reorder or insert in middle** — State will attach to the wrong item.
- **Never OK if items have state** — A text input in each row will keep its value attached to the wrong row after reorder.

9.4 Anti-Patterns

```
// [X] Math.random() as key – regenerates each render, no reuse at all
{items.map(item => <li key={Math.random()}>{item.text}</li>)}

// [X] Index as key when list reorders
{items.map((item, i) => <li key={i}>{item.text}</li>)}

// [X] Multiple elements with same key – React warns and behaves unpredictably
{items.map(item => <li key="item">{item.text}</li>)}

// OK Stable unique ID from data
{items.map(item => <li key={item.id}>{item.text}</li>)}
```

10. Forms

Forms are central to most apps — login, registration, search, settings. React handles forms via controlled components (React owns the input value via state) or uncontrolled components (DOM owns the value, React reads it on submit via a ref). Controlled is the recommended default.

10.1 Controlled Form

```
function ContactForm() {
  const [form, setForm] = useState({
    name: '', email: '', message: '', topic: 'general'
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Submit:', form);
    // POST to API
  };

  return (
```

```

    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} required />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
      <select name="topic" value={form.topic} onChange={handleChange}>
        <option value="general">General</option>
        <option value="sales">Sales</option>
        <option value="support">Support</option>
      </select>
      <textarea name="message" value={form.message} onChange={handleChange} />
      <button type="submit">Send</button>
    </form>
  );
}

```

10.2 Validation Patterns

```

function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  const validate = () => {
    const errs = {};
    if (!email) errs.email = 'Email is required';
    else if (!/^[@]+\.[^@]+\.[^@]+$/i.test(email)) errs.email = 'Invalid email';
    if (!password) errs.password = 'Password is required';
    else if (password.length < 8) errs.password = 'Min 8 chars';
    return errs;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    if (Object.keys(errs).length) {
      setErrors(errs);
      return;
    }
    setErrors({});
    // submit
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {errors.email && <span className="error">{errors.email}</span>}
      <input type="password" value={password}
        onChange={e => setPassword(e.target.value)} />
      {errors.password && <span className="error">{errors.password}</span>}
      <button type="submit">Login</button>
    </form>
  );
}

```

10.3 For Complex Forms — Use a Library

- **React Hook Form** — Best performance (uncontrolled under the hood), small bundle, easy validation via Zod/Yup.
- **Formik** — Older, controlled, larger bundle. Still common in legacy code.
- **TanStack Form** — Modern, framework-agnostic, headless.

- **Recommendation** — React Hook Form + Zod for new projects.

11. Lifting State Up

When two sibling components need to share or synchronise state, the React pattern is to "lift" the state to their common ancestor. The ancestor holds the state and passes it down via props along with setter callbacks. This is the canonical way to share state without a global store.

11.1 Example: Synchronised Inputs

```
// Without lifting: each input has its own state – they cannot sync.

// With lifting: parent owns the state
function SyncedInputs() {
  const [text, setText] = useState('');
  return (
    <div>
      <Input label="First" value={text} onChange={setText} />
      <Input label="Second" value={text} onChange={setText} />
    </div>
  );
}

function Input({ label, value, onChange }) {
  return (
    <label>
      {label}: { ' ' }
      <input value={value} onChange={e => onChange(e.target.value)} />
    </label>
  );
}
```

11.2 When to Lift State

- **Two siblings need the same data** — Move state to common parent.
- **Child needs to notify parent** — Pass a callback prop; parent updates state.
- **Multiple components reflect the same source of truth** — Single state in the parent; children receive via props.
- **Stop lifting when** — The chain is more than 2-3 levels deep with no intermediate component using the state — switch to Context or Redux Toolkit to avoid prop drilling.

12. Controlled vs Uncontrolled Components

A controlled input is one whose value is controlled by React state. An uncontrolled input keeps its value in the DOM, accessed via a ref. Controlled is the default and recommended pattern; uncontrolled is useful for one-time reads (file inputs, integrating with non-React code).

12.1 Comparison

Type	Source of Truth	Pattern	Use Case
Controlled	React state is the source of truth	value={state} onChange={setState}	Most forms
Uncontrolled	DOM is the source of truth	defaultValue + ref	File inputs, quick integrations

12.2 Controlled Example

```
function ControlledInput() {
  const [value, setValue] = useState('');
  return (
    <>
      <input value={value} onChange={e => setValue(e.target.value)} />
      <p>You typed: {value}</p>
    </>
  );
}

// Validation, formatting, conditional disabling – all easy because state
// is the source of truth.
```

12.3 Uncontrolled Example

```
import { useRef } from 'react';

function UncontrolledForm() {
  const inputRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Value:', inputRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input defaultValue="" ref={inputRef} />
      <button type="submit">Submit</button>
    </form>
  );
}

// File inputs are always uncontrolled (you cannot set their value)
<input type="file" ref={fileRef} />
```

12.4 When to Use Each

- **Controlled (default)** — Login forms, search boxes, settings — anywhere you need validation, formatting, or to read the value during render.
- **Uncontrolled** — File inputs (always), one-time read on submit, integrating with non-React libraries that manage their own DOM state, simple "good enough" forms where you do not need live validation.

- **React Hook Form** — Uses uncontrolled under the hood with refs for performance (fewer re-renders than controlled), but exposes a controlled-like API. Best of both worlds for big forms.

13. React Lifecycle

A React component goes through phases: mount (inserted into the DOM), update (state/props change), and unmount (removed from the DOM). Class components expose lifecycle methods (componentDidMount, etc.); function components use useEffect for the same purpose.

13.1 Lifecycle Phases

Phase	Trigger	Class / Function Equivalent
Mount	Component is added to the DOM	constructor → render → componentDidMount (class) / useEffect with [] (function)
Update	State or props change	render → componentDidUpdate / useEffect with deps
Unmount	Component is removed	componentWillUnmount / useEffect cleanup
Error	A child throws during render	componentDidCatch (class only)

13.2 Class Lifecycle Methods

Method	When	Use
constructor	Before mount	Initialise state, bind methods
getDerivedStateFromProps	Before render (mount & update)	Sync state with props (rarely needed)
render	Every render	Return JSX (required)
componentDidMount	After first render	Fetch data, add listeners, start timers
shouldComponentUpdate	Before re-render	Performance: return false to skip
getSnapshotBeforeUpdate	Before DOM update	Capture scroll position, etc.
componentDidUpdate	After re-render	Side effects based on prev props/state
componentWillUnmount	Before removal	Cleanup: remove listeners, clear timers
componentDidCatch	After child throws	Error boundary (class only)

13.3 Function Equivalent (useEffect)

```
import { useEffect, useState } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
```

```

// componentDidMount + componentDidUpdate (when userId changes)
// + componentWillUnmount
useEffect(() => {
  let cancelled = false;
  fetchUser(userId).then(u => { if (!cancelled) setUser(u); });

  // cleanup = componentWillUnmount (and before re-running effect)
  return () => { cancelled = true; };
}, [userId]); // dependency array

if (!user) return <Spinner />;
return <div>{user.name}</div>;
}

// Dep array variants:
// useEffect(fn) – runs every render (avoid)
// useEffect(fn, []) – runs once (mount only)
// useEffect(fn, [a, b]) – runs on mount and when a or b change

```

14. Functional Components vs Class Components

Before hooks (React 16.8, 2019), class components were the only way to use state and lifecycle. Hooks let function components do everything classes could, with simpler syntax. Today, function components are the default; classes are legacy.

14.1 Comparison

Aspect	Function Component	Class Component
Syntax	Plain function returning JSX	ES6 class with render()
State	useState hook	this.state + this.setState
Lifecycle	useEffect, useEffect	componentDidMount, etc.
this binding	No `this`	Must bind handlers or use arrow class fields
Hooks	All hooks	Cannot use hooks
Error boundaries	Not (until React 19)	Yes — componentDidCatch
Boilerplate	Less	More
Use today	Default	Legacy code, error boundaries (pre-React 19)

14.2 Side-by-Side: Counter

```

// Function component (modern)
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (

```

```

    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}

// Class component (legacy)
import { Component } from 'react';

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.handleClick = this.handleClick.bind(this); // required!
  }

  handleClick() {
    this.setState(prev => ({ count: prev.count + 1 }));
  }

  render() {
    return (
      <button onClick={this.handleClick}>
        Count: {this.state.count}
      </button>
    );
  }
}

```

14.3 Why Function Components Won

- **Simpler syntax** — No constructor, no this, no binding.
- **Hooks compose better** — Custom hooks can be extracted and reused across components; class mixins were a mess.
- **Less boilerplate** — A class counter is ~20 lines; a function counter is ~8.
- **Tree-shaking friendly** — Functions are easier for bundlers to eliminate than class methods.
- **Concurrent features** — React 18's concurrent rendering works transparently with hooks; classes need extra workarounds.
- **Same power** — Hooks (useState, useEffect, useContext, useRef, useMemo, useCallback, useReducer) cover everything classes could do.

14.4 When Classes Are Still Used

- **Error boundaries** — componentDidCatch is class-only — but React 19 may introduce function error boundaries, and the react-error-boundary library provides a HOC-style wrapper you can use without writing a class.
- **Legacy codebases** — Many existing apps still have class components. Migration to functions is gradual.
- **React Native pre-0.65** — Older RN versions had perf issues with function components; modern RN prefers functions.

15. Hooks

Hooks are functions that let you "hook into" React features (state, lifecycle, context) from function components. They were introduced in React 16.8 (2019) and replaced class lifecycle methods as the default way to write components.

15.1 Built-in Hooks

Hook	Purpose	When
useState	Local state	Re-render on change
useEffect	Side effects (fetch, subscriptions)	After render
useLayoutEffect	Synchronous side effects (DOM measurement)	Before browser paints
useContext	Read context value	Avoid prop drilling
useRef	Mutable container, persist across renders	DOM refs + values
useMemo	Memoise expensive computation	Skip recomputation
useCallback	Memoise function reference	Stable callback identity
useReducer	Reducer-style state (like Redux)	Complex state logic
useId	Stable unique ID for accessibility	Server/client consistent
useImperativeHandle	Customise ref exposed to parent	Wrap with forwardRef
useDebugValue	Show label in DevTools	Custom hook debugging
useDeferredValue (18+)	Defer a value for non-urgent updates	Concurrent rendering
useTransition (18+)	Mark update as non-urgent	Concurrent rendering
useSyncExternalStore (18+)	Subscribe to external store safely	State library internals

15.2 Rules of Hooks

- **Call at top level only** — Never inside loops, conditions, or nested functions. Hooks rely on call order — React tracks them by position.
- **Call from React functions only** — From function components or custom hooks (functions named use*). Never from regular JS functions.
- **eslint-plugin-react-hooks** — Enforces both rules. Enable in every React project — catches 90% of hook bugs.

15.3 Why Hooks?

- **Reuse logic** — Custom hooks let you share stateful logic across components without HOCs or render props.
- **Cleaner lifecycle** — `useEffect` groups related concerns; class lifecycle mixed them in `componentDidMount`.
- **No `this`** — Functions avoid the `this`-binding footguns of classes.
- **Smaller bundles** — Better tree-shaking; minified React is smaller.

16. useState

`useState` is the most-used hook. It declares a state variable and returns a tuple of `[currentValue, setter]`. Calling the setter with a new value schedules a re-render.

16.1 Basic Usage

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

16.2 Functional Updates

```
// [X] WRONG – uses stale `count` from closure
function increment() {
  setCount(count + 1); // 0 + 1
  setCount(count + 1); // 0 + 1 again (closure captured count = 0)
}
// Final count: 1 (instead of expected 2)

// OK RIGHT – functional update uses the latest state
function increment() {
  setCount(c => c + 1); // 0 + 1
  setCount(c => c + 1); // 1 + 1
}
// Final count: 2 OK
```

16.3 Lazy Initialisation

```
// [X] Expensive computation runs every render (result ignored)
const [data, setData] = useState(loadFromLocalStorage());

// OK Lazy init – function runs ONCE on first render
const [data, setData] = useState(() => loadFromLocalStorage());

// Use for: localStorage reads, large initial computations, anything
```

```
// expensive that you do not want to run on every render.
```

16.4 Object State — Always Replace, Never Mutate

```
const [form, setForm] = useState({ name: '', email: '', age: 0 });

// [X] WRONG – mutates existing state (React may not detect change)
form.name = 'Alice';
setForm(form);

// OK RIGHT – new object with spread
setForm({ ...form, name: 'Alice' });

// OK RIGHT – functional form (works with previous state)
setForm(prev => ({ ...prev, name: 'Alice' }));

// OK Best – use Immer for nested updates
import { useImmer } from 'use-immer';
const [form, setForm] = useImmer({ name: '', email: '', address: { city: '' } });
setForm(draft => { draft.address.city = 'NYC'; }); // mutates draft, React sees new object
```

16.5 When to Use useReducer Instead

- **Multiple related states** — If 3+ states change together, useReducer with one action is cleaner.
- **Complex next-state logic** — If the next state depends on multiple fields and conditions, a reducer is more testable.
- **Team conventions** — Redux/Redux Toolkit users find useReducer familiar.
- **Rule of thumb** — If you find yourself writing setX(prev => complex logic) repeatedly, switch to useReducer.

17. useEffect

useEffect runs side effects (data fetching, subscriptions, DOM manipulation, timers) after the component renders. It replaces componentDidMount, componentDidUpdate, and componentWillUnmount in class components.

17.1 Anatomy

```
useEffect(() => {
  // effect: runs after render
  console.log('count is', count);

  // cleanup: runs before next effect AND on unmount
  return () => {
    console.log('cleanup for count', count);
  };
}, [count]); // dependency array: effect runs when count changes

// Three forms of dependency array:
// useEffect(fn) – runs after EVERY render (rarely what you want)
// useEffect(fn, []) – runs ONCE on mount (no deps)
// useEffect(fn, [a, b]) – runs on mount and when a or b change
```

17.2 Common Patterns

```
// 1. Fetch data on mount
useEffect(() => {
  let cancelled = false;
  fetch(`/api/users/${userId}`)
    .then(r => r.json())
    .then(data => { if (!cancelled) setUser(data); });
  return () => { cancelled = true; }; // prevent setState on unmounted
}, [userId]);

// 2. Subscribe + unsubscribe
useEffect(() => {
  const handler = (e) => setMouse({ x: e.clientX, y: e.clientY });
  window.addEventListener('mousemove', handler);
  return () => window.removeEventListener('mousemove', handler);
}, []);

// 3. Timer
useEffect(() => {
  const id = setInterval(() => setTick(t => t + 1), 1000);
  return () => clearInterval(id);
}, []);

// 4. Sync document title
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

17.3 Cleanup Function

- **When it runs** — Before the next effect (when deps change) AND on unmount.
- **Why needed** — Prevent memory leaks, stale closures, and "setState on unmounted component" warnings.
- **Always return cleanup for** — Event listeners, subscriptions, timers, websockets, abort controllers.

17.4 Common Pitfalls

- **Missing dependency** — Forgetting to include a value used in the effect causes stale closure bugs. The eslint-plugin-react-hooks catches this.
- **Object/function dependency** — `setSearch({ term })` creates a new object every render, so the effect runs every render. Extract primitives: `useEffect(fn, [search.term])`.
- **Infinite loop** — `useEffect(() => setData(...))` without deps — runs on every render, which causes another render.
- **setState on unmounted** — Always use a cancelled flag or `AbortController` in async effects.
- **Effect for derived data** — Don't use `useEffect` to compute a value from props — compute during render or `useMemo` instead.

Modern tip (React 18+): For data fetching, prefer TanStack Query (React Query) over hand-rolled `useEffect`. It handles caching, deduplication, background refresh, retries, and cleanup automatically — all of which you would otherwise write in `useEffect`.

17.5 useEffect vs useLayoutEffect

Hook	Timing	Use
useEffect	After browser paints	Most side effects (default)
useLayoutEffect	Before browser paints (synchronous)	DOM measurement that must be applied before user sees flash

18. useContext

useContext reads a value from a React Context without prop drilling. Combined with createContext and a Provider, it lets you share app-wide data (theme, locale, current user) across deeply nested components.

18.1 Three Steps

```
// 1. Create a context (usually in a separate file)
import { createContext, useContext } from 'react';

const ThemeContext = createContext('light'); // default value

// 2. Provide a value at the top of the tree
function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Page />
    </ThemeContext.Provider>
  );
}

// 3. Consume the value in any descendant
function Toolbar() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

18.2 When to Use Context

- **Theme, locale, current user** — Rarely-changed global data.
- **Auth state** — Logged-in user, sign-out function.
- **Router state** — React Router uses Context internally.
- **DI for services** — API client, analytics, feature flags.
- **Avoid for high-frequency updates** — Every consumer re-renders when the Provider value changes. For 60fps updates (canvas, drag), use refs or a specialised library (Zustand, Jotai).

18.3 Context Performance

```
// [X] Every Toolbar re-render creates a new value object – all
// consumers re-render even if theme did not change.
function App() {
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

// OK Memoise the value
const value = useMemo(() => ({ theme, setTheme }), [theme]);
return (
  <ThemeContext.Provider value={value}>
    <Toolbar />
  </ThemeContext.Provider>
);

// OK Or split: context for value, separate for setter (which never changes)
<ThemeContext.Provider value={theme}>
  <ThemeSetContext.Provider value={setTheme}>
    <Toolbar />
  </ThemeSetContext.Provider>
</ThemeContext.Provider>
```

18.4 Custom Hook for Context

```
// Wrap useContext in a custom hook for ergonomics + error check
function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used within ThemeProvider');
  return ctx;
}

// Usage
function Button() {
  const { theme } = useTheme();
  return <button className={theme}>Click</button>;
}
```

19. useRef

useRef returns a mutable object whose .current property persists across renders. Unlike state, changing .current does NOT trigger a re-render. Two main uses: (1) accessing DOM elements, (2) storing mutable values that should not trigger re-renders.

19.1 DOM Access

```
import { useRef } from 'react';

function SearchBox() {
  const inputRef = useRef(null);

  const focus = () => inputRef.current?.focus();
  const clear = () => {
    if (inputRef.current) inputRef.current.value = '';
  };
}
```

```

};

return (
  <>
    <input ref={inputRef} type="text" />
    <button onClick={focus}>Focus</button>
    <button onClick={clear}>Clear</button>
  </>
);
}

```

19.2 Mutable Value Container

```

function Stopwatch() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null); // holds timer id

  const start = () => {
    if (intervalRef.current) return; // already running
    intervalRef.current = setInterval(() => setSeconds(s => s + 1), 1000);
  };

  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  };

  useEffect(() => () => clearInterval(intervalRef.current), []);

  return (
    <div>
      <p>{seconds}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}

```

19.3 useRef vs useState

Property	useState	useRef
Triggers re-render	Yes	No
Persistent across renders	Yes	Yes
Read in event handler	Latest value	Latest value
Read during render	current state	! stale (use inside effect)
Use for	UI state	DOM refs, timers, "previous value" tracking

19.4 Storing "Previous" Value

```
function Price({ price }) {
  const prevPriceRef = useRef(price);
  const prevPrice = prevPriceRef.current;

  useEffect(() => {
    prevPriceRef.current = price; // update AFTER render
  }, [price]);

  const direction = price > prevPrice ? 'up' : price < prevPrice ? 'down' : 'flat';
  return <span className={direction}>{price} ({direction})</span>;
}

// Or use a ready-made custom hook
function usePrevious(value) {
  const ref = useRef(value);
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current;
}
```

20. useMemo

useMemo memoises the result of an expensive computation, recomputing only when its dependencies change. Use sparingly — memoisation has its own cost, and the React team advises reaching for it only when profiling shows a real problem.

20.1 Basic Pattern

```
import { useMemo } from 'react';

function ProductList({ products, filter, sortKey }) {
  // Recompute only when products, filter, or sortKey change
  const visible = useMemo(() => {
    const filtered = products.filter(p => p.category === filter);
    return [...filtered].sort((a, b) => a[sortKey] - b[sortKey]);
  }, [products, filter, sortKey]);

  return (
    <ul>
      {visible.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}
```

20.2 When to Use

- **Expensive computation** — Sorting/filtering 1000+ items, complex math, parsing large JSON.
- **Referential equality for child props** — Passing an object/array to a memoised child — new ref each render would break memo. Use useMemo to keep the ref stable.
- **Avoiding re-renders in child via React.memo** — Combine useMemo for props + React.memo for the child to skip re-renders entirely.

20.3 When NOT to Use

- **Cheap operations** — Simple arithmetic, small array maps — the cost of `useMemo` (storing deps, comparing) exceeds the savings.
- **Premature optimisation** — Profile first with React DevTools. Many "optimisations" do not move the needle.
- **To work around a bug** — If your component re-renders too often, find the cause (new object in props, missing memo on parent) instead of bandaging with `useMemo`.

21. useCallback

`useCallback` returns a memoised version of a function. The function reference stays stable across re-renders unless its dependencies change. Useful for passing callbacks to memoised children or as effect dependencies.

21.1 Basic Pattern

```
import { useCallback, memo } from 'react';

function Parent({ data }) {
  const [filter, setFilter] = useState('');

  // Stable reference across renders unless `filter` changes
  const handleClick = useCallback((id) => {
    console.log('Selected:', id, 'with filter', filter);
  }, [filter]);

  return <MemoisedChild data={data} onSelect={handleClick} />;
}

const MemoisedChild = memo(function Child({ data, onSelect }) {
  console.log('Child rendered');
  return <ul>{data.map(d => <li key={d.id} onClick={() =>
onSelect(d.id)}>{d.name}</li>)}</ul>;
});

// Without useCallback, Parent creates a new handleClick every render,
// which breaks memo - Child re-renders even if `data` is unchanged.
```

21.2 When to Use

- **Passing callbacks to memoised children (React.memo)** — Without `useCallback`, the parent creates a new function every render, breaking memo.
- **Function as a `useEffect` dependency** — A new function reference every render would re-run the effect every render. Use `useCallback` (or move the function inside the effect) to stabilise.
- **Same function passed to multiple children** — Save memory by sharing one reference.

21.3 When NOT to Use

- **Non-memoised children** — If the child is not wrapped in `React.memo`, a stable callback does nothing — the child re-renders anyway.

- **Simple inline handlers** — `const onClick = () => doX();` on a non-critical button is fine. Premature `useCallback` adds noise.
- **Every callback** — Do not wrap every function — only the ones that would cause measurable problems. Profile first.

21.4 useCallback vs useMemo

```
// These are equivalent:
const handleClick = useCallback(() => { doSomething(x); }, [x]);
const handleClick = useMemo(() => () => { doSomething(x); }, [x]);

// useCallback is just sugar for the useMemo form. Use useCallback for
// readability when you specifically need a function reference.
```

22. useReducer

`useReducer` is an alternative to `useState` for complex state logic. You define a reducer function (current state, action) → new state, and dispatch actions instead of calling setters. Pattern matches Redux; useful when state transitions are interconnected.

22.1 Counter with useReducer

```
import { useReducer } from 'react';

const initial = { count: 0, lastAction: 'init' };

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { ...state, count: state.count + 1, lastAction: 'increment' };
    case 'decrement': return { ...state, count: state.count - 1, lastAction: 'decrement' };
    case 'reset': return { ...initial, lastAction: 'reset' };
    case 'set': return { ...state, count: action.value, lastAction: 'set' };
    default: throw new Error(`Unknown action: ${action.type}`);
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initial);
  return (
    <div>
      <p>Count: {state.count} (last: {state.lastAction})</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
      <button onClick={() => dispatch({ type: 'set', value: 100 })}>Set 100</button>
    </div>
  );
}
```

22.2 useState vs useReducer

Use Case	Choose	Why
Simple independent values	useState	const [n, setN] = useState(0)
Multiple related fields	useReducer	One action updates several
Complex next-state logic	useReducer	Testable reducer function
Next state depends on prev	Either	useState functional form OR useReducer
Transition logging / devtools	useReducer	Dispatch is auditable

23. Custom Hooks

A custom hook is a function whose name starts with "use" and that may call other hooks. Custom hooks let you extract component logic into reusable functions — the React replacement for HOCs and render props.

23.1 Anatomy

```
import { useState, useEffect } from 'react';

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handler = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handler);
    return () => window.removeEventListener('resize', handler);
  }, []);

  return width;
}

// Usage
function ResponsiveHeader() {
  const width = useWindowWidth();
  return <header>{width < 768 ? <MobileNav /> : <DesktopNav />}</header>;
}
```

23.2 Common Custom Hooks

```
// Fetch data with loading + error states
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch(url)
      .then(r => { if (!r.ok) throw new Error(r.statusText); return r.json(); })
      .then(d => { if (!cancelled) { setData(d); setError(null); } })
      .catch(e => { if (!cancelled) setError(e.message); })
  });
}
```

```

        .finally(() => { if (!cancelled) setLoading(false); });
    return () => { cancelled = true; };
}, [url]);

return { data, loading, error };
}

// localStorage-backed state
function useLocalStorage(key, initial) {
    const [value, setValue] = useState(() => {
        const stored = localStorage.getItem(key);
        return stored ? JSON.parse(stored) : initial;
    });

    useEffect(() => {
        localStorage.setItem(key, JSON.stringify(value));
    }, [key, value]);

    return [value, setValue];
}

// Debounce a value
function useDebounce(value, delay = 300) {
    const [debounced, setDebounced] = useState(value);
    useEffect(() => {
        const id = setTimeout(() => setDebounced(value), delay);
        return () => clearTimeout(id);
    }, [value, delay]);
    return debounced;
}

```

23.3 Rules

- **Name starts with `use`** — `useWindowSize`, `useFetch`, `useAuth`. The linter relies on this prefix to enforce rules of hooks.
- **Can call other hooks** — Custom hooks can call `useState`, `useEffect`, `useContext`, or other custom hooks freely.
- **Return anything** — Return a value, an array (like `useState`), or an object — pick what consumers will find ergonomic.
- **One concern per hook** — Don't cram multiple unrelated features into one hook. `useAuth`, `useTheme`, `useFetchUser` — separate.

24. React Router

React Router is the de-facto routing library for React SPAs. It maps URLs to components, supports nested routes, URL parameters, query strings, navigation guards, and lazy loading. The current version is React Router 7 (merged with Remix).

24.1 Basic Setup

```
import { createBrowserRouter, RouterProvider, Link, Outlet } from 'react-router-dom';

function Layout() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/products">Products</Link>
        <Link to="/about">About</Link>
      </nav>
      <Outlet /> {/* nested route renders here */}
    </div>
  );
}

const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Home /> },
      { path: 'products', element: <ProductList /> },
      { path: 'products/:id', element: <ProductDetail /> },
      { path: 'about', element: <About /> },
      { path: '*', element: <NotFound /> }
    ]
  }
]);

function App() {
  return <RouterProvider router={router} />;
}
```

24.2 URL Parameters

```
import { useParams, Link } from 'react-router-dom';

function ProductDetail() {
  const { id } = useParams(); // URL: /products/42 → id = "42"
  const { data, loading } = useFetch(`/api/products/${id}`);
  if (loading) return <Spinner />;
  return (
    <div>
      <Link to="/products">Back to list</Link>
      <h1>{data.name}</h1>
    </div>
  );
}
```

24.3 Navigation

```
import { useNavigate, Navigate, useLocation } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from || '/';

  const handleSubmit = async (e) => {
```

```

    e.preventDefault();
    const ok = await login(...);
    if (ok) navigate(from, { replace: true }); // redirect after login
  };
  // ...
}

// Programmatic redirect (declarative)
function ProtectedPage({ user }) {
  if (!user) return <Navigate to="/login" state={{ from: location.pathname }} replace
/>;
  return <Dashboard />;
}

```

24.4 Lazy-Loaded Routes

```

import { lazy, Suspense } from 'react';

const ProductList = lazy(() => import('./ProductList'));
const ProductDetail = lazy(() => import('./ProductDetail'));
const AdminPanel = lazy(() => import('./AdminPanel'));

const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Home /> },
      {
        path: 'products',
        element: <Suspense fallback={<Spinner />}><ProductList /></Suspense>
      },
      // ... other lazy routes
    ]
  }
]);

```

25. State Management

State management is how you store, update, and share data across components. React ships with `useState`, `useReducer`, and `Context` — enough for small apps. Larger apps add a state library (Redux Toolkit, Zustand, Jotai) for predictable updates and selective subscriptions.

25.1 State Categories

Category	Scope	Tool
Local	Single component	<code>useState</code> , <code>useReducer</code>
Shared (sibling tree)	Few components	Lift state up to common parent
Global, low-frequency	Whole app, rarely changes	<code>Context</code> (theme, user)
Global, high-frequency	Whole app, often changes	Redux Toolkit, Zustand

Category	Scope	Tool
Server state	Cached server data	TanStack Query, SWR
URL state	Filters, pagination	URL params + React Router

25.2 Choose the Right Tool

- **Start local** — useState/useReducer for component state. Do not reach for a global store first.
- **Lift state up** — For 2-3 sibling components sharing state, lift to common parent.
- **Context for ambient data** — Theme, locale, current user — rarely changes, consumed widely.
- **Redux Toolkit for complex apps** — When you need time-travel debugging, predictable updates, or have many slices of business logic.
- **Zustand for lightweight global** — Simple API, no boilerplate, selector subscriptions — modern alternative to Redux for most apps.
- **TanStack Query for server data** — Do NOT put server-fetched data in Redux. Use TanStack Query for caching, refetching, mutations.

26. Redux Toolkit

Redux Toolkit (RTK) is the official, recommended way to write Redux. It replaces hand-written action types, action creators, and reducers with configureStore + createSlice, cutting boilerplate by ~80%. RTK Query (built-in) handles data fetching.

26.1 Setup

```
// store.ts
import { configureStore, createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload); // "mutate" - Immer makes it immutable
      state.total = state.items.reduce((s, i) => s + i.price, 0);
    },
    removeItem: (state, action) => {
      state.items = state.items.filter(i => i.id !== action.payload);
      state.total = state.items.reduce((s, i) => s + i.price, 0);
    },
    clear: (state) => {
      state.items = [];
      state.total = 0;
    }
  }
});

export const { addItem, removeItem, clear } = cartSlice.actions;

export const store = configureStore({
  reducer: { cart: cartSlice.reducer }
});
```

```
});
```

26.2 Using in Components

```
import { useSelector, useDispatch } from 'react-redux';
import { addItem } from './store';

function ProductCard({ product }) {
  const dispatch = useDispatch();
  const totalItems = useSelector(state => state.cart.items.length);

  return (
    <div>
      <h3>{product.name} - ${product.price}</h3>
      <button onClick={() => dispatch(addItem(product))}>Add to cart</button>
      <p>Cart has {totalItems} items</p>
    </div>
  );
}

// Wrap app in Provider
import { Provider } from 'react-redux';
root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

26.3 Async with createAsyncThunk

```
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchProducts = createAsyncThunk(
  'products/fetchAll',
  async (_, thunkAPI) => {
    const res = await fetch('/api/products');
    if (!res.ok) return thunkAPI.rejectWithValue(res.statusText);
    return res.json();
  }
);

const productsSlice = createSlice({
  name: 'products',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.payload;
      });
  }
});
```

Modern tip: For server data (fetch, cache, invalidate), prefer RTK Query (built into Redux Toolkit) or TanStack Query over createAsyncThunk. They handle caching, deduplication, retries, and background refetch automatically — saving you from writing that boilerplate for every endpoint.

27. Context API

Context API is React's built-in way to share values across the component tree without prop drilling. A Provider publishes a value; any descendant can read it via useContext. Covered earlier in topic 18 — this section focuses on patterns and best practices.

27.1 Provider Pattern

```
// ThemeContext.tsx
import { createContext, useContext, useState, useMemo } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() =>
    localStorage.getItem('theme') || 'light'
  );

  const value = useMemo(() => ({
    theme,
    toggle: () => setTheme(t => t === 'light' ? 'dark' : 'light'),
    set: setTheme
  }), [theme]);

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used within ThemeProvider');
  return ctx;
}
```

27.2 Splitting Contexts

A single big context forces every consumer to re-render on any change. Split into separate contexts by update frequency or concern.

27.3 Context vs Redux

Aspect	Context API	Redux Toolkit
Use case	Global app-wide rarely-changing data	Complex business state with frequent updates

Aspect	Context API	Redux Toolkit
Boilerplate	Low	Medium (RTK cuts it)
Performance	All consumers re-render on change	Selector subscriptions — only affected components
DevTools	None built-in	Time-travel, action log
Async	Manual (useEffect + setState)	createAsyncThunk, RTK Query
Middleware	None	Redux Thunk, Saga, Logger

28. Performance Optimization

React is fast by default — most "performance issues" are caused by unnecessary re-renders, large bundles, or expensive operations in render. Optimise only after profiling; do not guess.

28.1 Tools

- **React DevTools Profiler** — Record a session, see which components rendered and why (state change, parent render, context change).
- **why-did-you-render** — Logs unnecessary re-renders in dev. npm install @welldone-software/why-did-you-render
- **Lighthouse** — Audit bundle size, LCP, TTI, accessibility.
- **Chrome Performance tab** — CPU profile, flame chart.
- **React 19 DevTools** — Improved "Rendered at" and "Re-rendered due to" indicators.

28.2 Optimisation Techniques

Technique	What It Does	Use For
React.memo	Skip re-render if props are shallow-equal	Pure components
useMemo	Memoise expensive computation	Filtering/sorting large lists
useCallback	Stable function reference	Callbacks passed to memoised children
Lazy load	Split code per route	React.lazy + Suspense
Virtualization	Render only visible rows	Large lists (react-window, react-virtual)
useTransition	Mark update as non-urgent	Search results, filtering
useDeferredValue	Defer a value	Type-ahead search
Key props	Stable identity for list items	Lists that reorder

Technique	What It Does	Use For
Windowing	Paginate / infinite scroll	Long lists
Code splitting	Dynamic import	Large libraries used on one page

28.3 Common Pitfalls

- **Premature memoization** — Wrapping every component in `React.memo` without measuring — the cost of shallow compare may exceed savings.
- **Inline objects/functions as props** — `style={{ color: red }}` creates a new object each render — breaks memo on the child.
- **Reconciliation overhead** — Re-rendering is cheap; committing to DOM is expensive. Memo helps only when the rendered output is unchanged.
- **Big bundle** — First load matters more than re-renders. Audit with `webpack-bundle-analyzer` or `vite-bundle-visualizer`.
- **Stale closures** — `useEffect` missing deps cause stale data — fix with `exhaustive-deps` lint rule, not by skipping deps.

29. React.memo

`React.memo` is a higher-order component that memoises a function component. If the props are shallow-equal to the previous render, React skips re-rendering and reuses the last output. Useful for pure components in trees that re-render often.

29.1 Basic Usage

```
const ExpensiveList = memo(function ExpensiveList({ items, filter }) {
  console.log('ExpensiveList rendered');
  return (
    <ul>
      {items.filter(i => i.category === filter).map(i => (
        <li key={i.id}>{i.name}</li>
      ))}
    </ul>
  );
});

// Now if parent re-renders but items and filter are unchanged, this
// component skips re-render.
```

29.2 Custom Comparator

```
// Default: shallow compare props. Override with a custom comparator
// that returns true if props are equal (skip render), false otherwise.
const MyComponent = memo(function MyComponent(props) {
  return <div>{props.user.name}</div>;
}, (prev, next) => {
  // Only re-render if user.id changed (ignore other fields)
  return prev.user.id === next.user.id;
});
```

29.3 When to Use

- **Pure components in frequently-re-rendering parents** — Parent updates often; child only cares about a few props.
- **Lists with many items** — Memoise each item; only changed items re-render.
- **Expensive render** — Component does heavy computation or has many DOM nodes.

29.4 When NOT to Use

- **Always-on rendering** — If the parent always changes the props, memo just adds overhead.
- **Simple components** — For a button with no children, the shallow compare may cost more than the render.
- **When props are always new objects** — memo is useless if parent creates new array/object props each render — fix the parent first.

30. Lazy Loading

Lazy loading defers loading a component's code until it is needed. Combined with code splitting (the bundler creates a separate chunk per dynamic import), this dramatically reduces initial bundle size and improves first paint.

30.1 React.lazy

```
import { lazy, Suspense } from 'react';

// Lazy-load the AdminPanel — only fetched when first rendered
const AdminPanel = lazy(() => import('./AdminPanel'));
const Reports = lazy(() => import('./Reports'));
const Settings = lazy(() => import('./Settings'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Router>
        <Routes>
          <Route path="/admin" element={<AdminPanel />} />
          <Route path="/reports" element={<Reports />} />
          <Route path="/settings" element={<Settings />} />
        </Routes>
      </Router>
    </Suspense>
  );
}

// Bundler (Vite/webpack) creates separate JS chunks for each import().
// Initial bundle stays small; admin code is only fetched when user
// navigates to /admin.
```


30.2 Lazy Loading Routes

- **Per-route splitting** — Most impactful — each route is a separate chunk. User on /products does not download /admin code.
- **Per-feature splitting** — Heavy features (chart library, rich text editor) loaded only when feature opens.
- **Above-the-fold vs below** — Load critical content first; defer below-fold content with Intersection Observer.
- **Prefetch** — Hint the browser to fetch a chunk in idle time: `<link rel="prefetch" href="..." />`. Vite does this automatically for adjacent routes.

31. Suspense

Suspense lets a component "suspend" — pause rendering and show a fallback UI — while waiting for async work (lazy-loaded code, data fetching via React 19 / TanStack Query, or streaming server components). Suspense is the unified loading state for React.

31.1 Basic Suspense

```
import { Suspense, lazy } from 'react';

const UserProfile = lazy(() => import('./UserProfile'));

function App() {
  return (
    <Suspense fallback={<div>Loading profile...</div>}>
      <UserProfile userId={42} />
    </Suspense>
  );
}
```

31.2 Nested Suspense

```
// Outer fallback shows until header loads; inner fallbacks show
// independently as each section resolves.
<Suspense fallback={<PageLoader />}>
  <Header />
  <main>
    <Suspense fallback={<Spinner label="Profile" />}>
      <UserProfile />
    </Suspense>
    <Suspense fallback={<Spinner label="Feed" />}>
      <Feed />
    </Suspense>
  </main>
</Suspense>
```

31.3 Suspense for Data Fetching (React 19 / TanStack Query)

```
// With TanStack Query's useSuspenseQuery (or React 19's use() hook):
function ProductList() {
  const { data } = useSuspenseQuery({
    queryKey: ['products'],
    queryFn: () => fetch('/api/products').then(r => r.json())
  });
}
```

```

    });
    // data is guaranteed defined here – Suspense handles loading
    return data.map(p => <Product key={p.id} {...p} />);
  }

  // Parent wraps in Suspense:
  <Suspense fallback={<Spinner />}>
    <ProductList />
  </Suspense>

```

32. Error Boundaries

An error boundary is a React component that catches errors in its child tree during render, lifecycle, or constructors — preventing the whole app from unmounting. They show a fallback UI instead of a white screen. Class-based (componentDidCatch); React 19 introduces function error boundaries, but the `react-error-boundary` library is still common.

32.1 Class-Based Error Boundary

```

import { Component } from 'react';

class ErrorBoundary extends Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, info) {
    console.error('Caught:', error, info);
    // Send to Sentry, LogRocket, etc.
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback || <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary fallback={<ErrorPage />}>
  <App />
</ErrorBoundary>

```

32.2 `react-error-boundary` (Recommended)

```

import { ErrorBoundary } from 'react-error-boundary';

function App() {
  return (
    <ErrorBoundary
      fallbackComponent={ErrorFallback}
      onError={(error, info) => Sentry.captureException(error)}
      onReset={() => window.location.reload()}
      resetKeys={[location.pathname]}
    >

```

```
    >
    <Routes />
  </ErrorBoundary>
);
}

function ErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert">
      <h2>Something went wrong</h2>
      <pre>{error.message}</pre>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}
```

32.3 What Error Boundaries Do NOT Catch

- **Event handlers** — onClick handlers — wrap in try/catch manually.
- **Async code** — setTimeout, requestAnimationFrame, promise .then — try/catch or .catch().
- **Server-side rendering** — Errors during SSR — log and redirect.
- **Errors thrown in the error boundary itself** — It cannot catch its own bugs.

33. Portals, HOCs, Render Props, Compound Components

These are advanced patterns that you may not write daily but should recognise in code and explain in interviews. Modern React favours hooks and composition, but these patterns remain useful for specific cases.

33.1 Portals

createPortal lets you render a component into a different DOM node (typically a direct child of document.body). Use for modals, tooltips, popovers — anything that must escape parent stacking context, overflow: hidden, or z-index issues.

```
import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;
  return createPortal(
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.body // render target outside React root
  );
}
```

33.2 Higher-Order Components (HOCs)

An HOC is a function that takes a component and returns a new component with added props or behaviour. HOCs were the main reuse pattern before hooks; modern code uses custom hooks instead. Still common in legacy codebases (especially with Redux's connect).

```
// HOC: adds "user" prop from context
function withUser(Component) {
  return function WrappedComponent(props) {
    const { user } = useAuth();
    return <Component {...props} user={user} />;
  };
}

const Profile = withUser(function Profile({ user }) {
  return <h1>{user.name}</h1>;
});

// Modern equivalent: just use the hook inside the component
function Profile() {
  const { user } = useAuth();
  return <h1>{user.name}</h1>;
}
```

33.3 Render Props

A render prop is a prop (often called `render` or `children`) that is a function the component calls to render part of its UI. Like HOCs, render props are largely replaced by hooks but appear in legacy code.

```
// Mouse tracker as a render-prop component
class MouseTracker extends React.Component {
  state = { x: 0, y: 0 };
  handleMouseMove = (e) => this.setState({ x: e.clientX, y: e.clientY });
  render() {
    return (
      <div onMouseMove={this.handleMouseMove}>
        {this.props.render(this.state)}
      </div>
    );
  }
}

// Usage
<MouseTracker render={({ x, y }) => <p>Position: {x}, {y}</p> />

// Modern equivalent: custom hook
function useMousePosition() {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => {
    const handler = e => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener('mousemove', handler);
    return () => window.removeEventListener('mousemove', handler);
  }, []);
  return pos;
}
```

33.4 Compound Components

Compound components are multiple components designed to be used together with a shared implicit state. Examples: Select + Option, Tabs + Tab, Accordion + Item. The pattern exposes a clean API ("Select with Option children") without prop drilling.

```
// Using React Context to share state among compound components
const TabsContext = createContext();

function Tabs({ defaultIndex = 0, children }) {
  const [active, setActive] = useState(defaultIndex);
  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

function TabList({ children }) {
  return <div className="tab-list">{children}</div>;
}

function Tab({ index, children }) {
  const { active, setActive } = useContext(TabsContext);
  return (
    <button
      className={active === index ? 'active' : ''}
      onClick={() => setActive(index)}
    >
      {children}
    </button>
  );
}

function TabPanel({ index, children }) {
  const { active } = useContext(TabsContext);
  if (active !== index) return null;
  return <div className="tab-panel">{children}</div>;
}

Tabs.TabList = TabList;
Tabs.Tab = Tab;
Tabs.Panel = TabPanel;

// Usage – clean API
<Tabs defaultIndex={0}>
  <Tabs.TabList>
    <Tabs.Tab index={0}>Profile</Tabs.Tab>
    <Tabs.Tab index={1}>Settings</Tabs.Tab>
  </Tabs.TabList>
  <Tabs.Panel index={0}><ProfileForm /></Tabs.Panel>
  <Tabs.Panel index={1}><SettingsForm /></Tabs.Panel>
</Tabs>
```

34. Styling in React

React does not prescribe a styling solution. Common options include plain CSS, CSS Modules, CSS-in-JS (styled-components, Emotion), utility-first CSS (Tailwind), and component libraries (Material UI, Mantine, Chakra). Each makes different trade-offs between scoping, developer ergonomics, runtime cost, and bundle size.

34.1 Options at a Glance

Approach	How	Pros	Cons
Plain CSS	Global stylesheet	Simplest, no tooling	No scoping; class name clashes
CSS Modules	Local-scope by default	No runtime cost	Boilerplate import
CSS-in-JS (styled-components)	Write CSS in JS, scoped to component	Dynamic styling, props-driven	Runtime cost, larger bundle
Tailwind CSS	Utility classes in JSX	No custom CSS file, consistent design	Verbose className
Component library	Pre-built MUI/Chakra components	Fast MVP	Hard to customise; bundle size

34.2 Inline Styles

```
// Quick but limited – no pseudo-classes, no media queries
function Button({ color }) {
  return (
    <button style={{
      backgroundColor: color,
      padding: '8px 16px',
      borderRadius: 4,
      ':hover': { opacity: 0.8 } // [X] does not work
    }}>
      Click
    </button>
  );
}

// Use inline only for dynamic, computed styles. For static styles,
// use CSS classes (CSS Modules or Tailwind).
```

35. CSS Modules

CSS Modules are CSS files where class names are scoped locally by default. The build tool (Vite, webpack) rewrites class names to be unique, preventing global clashes. Supported out of the box in Vite and Create React App.

35.1 Usage

```
// Button.module.css
/* .button class is scoped to this file */
.button {
  background: #007bff;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
}
.button:hover { background: #0056b3; }
.primary { background: #28a745; }
.primary:hover { background: #1e7e34; }

// Button.tsx
import styles from './Button.module.css';

function Button({ variant = 'default', children, ...rest }) {
  return (
    <button
      className={` ${styles.button} ${styles[variant]} || ''`}
      {...rest}
    >
      {children}
    </button>
  );
}

// Compiled output: <button class="Button_button__3a8K2 Button_primary__9xLp1">
```

35.2 Pros & Cons

- **Pros** — Zero runtime cost, scoped class names, no naming collisions, works with TypeScript (with declaration).
- **Cons** — Two files per component (TSX + CSS), no dynamic styling based on props (need className conditionals), no Sass-like nesting without extra setup.

36. Styled Components

styled-components is a CSS-in-JS library that lets you write actual CSS inside your JavaScript, scoped to a styled component. Styles can be dynamic (based on props). It runs at runtime — every styled component injects a `<style>` tag with the compiled CSS.

36.1 Basic Usage

```
import styled from 'styled-components';

const Button = styled.button`
  background: ${props => props.primary ? '#28a745' : '#007bff'};
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  &:hover { opacity: 0.9; }
  &.disabled { opacity: 0.5; cursor: not-allowed; }
```

```
`;  
  
// Usage  
<Button primary>Save</Button>  
<Button>Cancel</Button>  
<Button className="disabled" disabled>Submit</Button>
```

36.2 Pros & Cons

- **Pros** — Dynamic styles via props, full Sass-like syntax, no class name clashes, dead-code elimination (unused styles removed with the component).
- **Cons** — Runtime overhead (parsing + injecting CSS), larger bundle (~12 KB gz), harder to debug (generated class names), no SSR streaming without setup.
- **Alternative** — Emotion (similar API, slightly faster). vanilla-extract / Linaria (zero-runtime, compiled at build time).

37. Tailwind CSS

Tailwind is a utility-first CSS framework. Instead of writing custom CSS classes, you compose utility classes directly in JSX: `className="bg-blue-500 text-white p-4 rounded"`. The result is no separate CSS file (besides Tailwind's generated utility classes), consistent design tokens, and tiny production CSS (only used classes are generated).

37.1 Basic Usage

```
function Button({ variant = 'default', children, ...rest }) {  
  const base = 'px-4 py-2 rounded font-medium transition-colors';  
  const variants = {  
    default: 'bg-blue-500 hover:bg-blue-600 text-white',  
    primary: 'bg-green-500 hover:bg-green-600 text-white',  
    outline: 'border border-gray-300 hover:bg-gray-50 text-gray-700'  
  };  
  return (  
    <button className={` ${base} ${variants[variant]} `} {...rest}>  
      {children}  
    </button>  
  );  
}
```

```
// Usage  
<Button>Default</Button>  
<Button variant="primary">Save</Button>  
<Button variant="outline">Cancel</Button>
```

37.2 Pros & Cons

- **Pros** — No custom CSS file, consistent design system, tiny production CSS (~10 KB), responsive variants (md:px-8 lg:px-12), dark mode (dark:bg-gray-900), excellent IDE autocomplete.
- **Cons** — Verbose className strings, learning curve for utility names, looks ugly to some developers, hard to extract complex components (use @apply or component extraction).

- **With React** — Popular combo: Tailwind + headless UI library (Radix UI, Headless UI) for accessible interactive components without opinionated styling.

38. API Calls

React does not prescribe how to call APIs. The native fetch API works, but most production apps use a library (Axios, TanStack Query) for ergonomics, interceptors, and caching.

38.1 Fetching in useEffect

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const ac = new AbortController();
    setLoading(true);
    fetch(`/api/users/${userId}`, { signal: ac.signal })
      .then(r => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json();
      })
      .then(d => { setUser(d); setError(null); })
      .catch(e => {
        if (e.name !== 'AbortError') setError(e.message);
      })
      .finally(() => setLoading(false));
    return () => ac.abort();
  }, [userId]);

  if (loading) return <Spinner />;
  if (error) return <ErrorView message={error} />;
  return <div>{user.name}</div>;
}
```

38.2 Why You Should Use TanStack Query Instead

- **Caching** — Same query called from 5 components → 1 network request.
- **Background refetch** — Refetch on window focus, on reconnect, after mutation — configurable.
- **Deduplication** — Multiple components querying the same key share one request.
- **Mutations** — useMutation with optimistic updates, rollback on error.
- **Retry / backoff** — Built-in retry with exponential backoff.
- **Devtools** — Visualise cache, refetch manually, inspect requests.
- **Persistence** — Optional cache persistence to localStorage.

Modern recommendation: For most production apps, use TanStack Query (formerly React Query) for server state. Skip hand-rolled useEffect + fetch + useState patterns except in trivial cases. The caching, refetching, and deduplication alone are worth it.

39. Axios vs Fetch

fetch is the native browser API for HTTP requests. Axios is a popular third-party library that adds interceptors, JSON parsing, timeouts, and better error handling. Both work; pick based on needs.

39.1 Comparison

Feature	fetch	Axios
JSON parsing	Manual: res.json()	Automatic
Error handling	Rejects only on network error; 4xx/5xx are success	Rejects on non-2xx automatically
Interceptors	No	Yes — auth headers, logging, retry
Timeout	Manual via AbortController	timeout: 5000 option
Cancel	AbortController (modern)	CancelToken (legacy) + AbortController
Progress events	No (upload), yes (download)	Yes for both
Bundle	0 KB (native)	~12 KB gz
Browser support	All modern browsers	All modern browsers + Node

39.2 Fetch Example

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

39.3 Axios Example

```
import axios from 'axios';

const api = axios.create({
  baseURL: '/api',
  timeout: 5000,
  headers: { 'Content-Type': 'application/json' }
});

// Request interceptor – attach auth token
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor – handle 401
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {

```

```

    localStorage.removeItem('token');
    window.location.href = '/login';
  }
  return Promise.reject(err);
}
);

// Usage
const { data } = await api.get(`/users/${id}`); // JSON parsed, 401 handled

```

40. Authentication

Frontend auth has three parts: storing the token (memory, localStorage, or httpOnly cookie), attaching it to API requests (Authorization header or cookie), and protecting routes (redirect unauthenticated users to login).

40.1 Token Storage Trade-offs

Storage	Pros	Cons
In-memory (state)	XSS-safe	Lost on refresh
localStorage	Survives refresh	XSS can read it
httpOnly cookie	XSS-safe (JS cannot read)	CSRF risk; needs credentialed fetch
SessionStorage	Cleared on tab close	Same XSS risk as localStorage

40.2 Auth Context Pattern

```

// AuthContext.tsx
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(() => localStorage.getItem('token'));

  const login = async (email, password) => {
    const res = await fetch('/api/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });
    if (!res.ok) throw new Error('Login failed');
    const { token, user } = await res.json();
    localStorage.setItem('token', token);
    setToken(token);
    setUser(user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
    setUser(null);
  };
}

```

```

    return (
      <AuthContext.Provider value={{ user, token, login, logout }}>
        {children}
      </AuthContext.Provider>
    );
  }

  export function useAuth() {
    const ctx = useContext(AuthContext);
    if (!ctx) throw new Error('useAuth must be used within AuthProvider');
    return ctx;
  }

```

41. JWT Authentication

JSON Web Token (JWT) is the most common stateless auth mechanism for SPAs. The server issues a signed token on login; the client sends it in the Authorization header on every request. The server verifies the signature and reads the user identity — no server-side session store needed.

41.1 JWT Structure

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikp5aWNlIiwiaWF0Ij0iMTUxMjM0NTY3fQ.signature

// Three Base64-URL-encoded parts separated by dots:
// 1. Header: { "alg": "HS256", "typ": "JWT" }
// 2. Payload: { "sub": "1234567890", "name": "Alice", "exp": 1701234567 }
// 3. Signature: HMAC-SHA256 of (header + "." + payload) using server secret

// ! Anyone can decode a JWT (it is just Base64). The signature only
// prevents tampering – do not store secrets in the payload.

```

41.2 Client-side JWT Flow

- **Login** — POST /api/login with credentials → receive access token (short-lived, 5-15 min) and refresh token (long-lived, days/weeks).
- **Store** — Access token in memory or localStorage; refresh token in httpOnly cookie (safer).
- **Attach** — Authorization: Bearer header on every API call.
- **Expire** — When access token expires, call /api/refresh with the refresh token to get a new one.
- **Logout** — Call /api/logout (invalidates refresh token server-side), clear tokens from client.

42. Protected Routes

A protected route is one that requires authentication. If the user is not logged in, redirect them to /login (with a "from" location state so they return after auth).

42.1 Pattern

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from './AuthContext';

function ProtectedRoute({ children }) {
  const { user } = useAuth();
  const location = useLocation();

  if (!user) {
    // Redirect to /login, remember where they were trying to go
    return <Navigate to="/login" state={{ from: location.pathname }} replace />;
  }
  return children;
}

// Usage in router
<Route
  path="/dashboard"
  element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  }
/>

// Login form: redirect back to original page after success
function LoginForm() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from || '/';

  const handleSubmit = async (e) => {
    e.preventDefault();
    await login(email, password);
    navigate(from, { replace: true });
  };
  // ...
}
```

42.2 Role-Based Protection

```
function RoleRoute({ children, roles }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (!roles.includes(user.role)) return <Navigate to="/forbidden" replace />;
  return children;
}

<Route path="/admin" element={<RoleRoute roles={['admin']}><AdminPanel /></RoleRoute>}
/>
```

43. React Query / TanStack Query

TanStack Query (formerly React Query) is the standard library for managing server state in React. It handles caching, refetching, pagination, optimistic updates, and deduplication — replacing hand-rolled `useEffect` + `useState` data fetching.

43.1 Setup

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000, // 1 min before refetch
      cacheTime: 5 * 60_000, // 5 min in cache after unmount
      retry: 3,
      refetchOnWindowFocus: true
    }
  }
});

root.render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

43.2 Basic Query

```
import { useQuery } from '@tanstack/react-query';

function ProductList() {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['products'], // cache key
    queryFn: () => fetch('/api/products').then(r => r.json())
  });

  if (isLoading) return <Spinner />;
  if (error) return <ErrorView message={error.message} />;
  return (
    <ul>
      {data.map(p => <li key={p.id}>{p.name}</li>)}
      <button onClick={() => refetch()}>Refresh</button>
    </ul>
  );
}

// Multiple components calling useQuery(['products']) share ONE request
// and the cached data. Switching back to this component shows cached
// data instantly and refetches in the background if stale.
```

43.3 Mutations

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function AddProduct() {
  const queryClient = useQueryClient();
```

```

const mutation = useMutation({
  mutationFn: (newProduct) =>
    fetch('/api/products', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(newProduct)
    }).then(r => r.json()),
  onSuccess: () => {
    // Invalidate the products list – will refetch
    queryClient.invalidateQueries({ queryKey: ['products'] });
  }
});

const handleSubmit = (e) => {
  e.preventDefault();
  mutation.mutate({ name: 'New Product', price: 9.99 });
};

return (
  <form onSubmit={handleSubmit}>
    <button disabled={mutation.isPending}>
      {mutation.isPending ? 'Adding...' : 'Add Product'}
    </button>
    {mutation.isError && <p>Error: {mutation.error.message}</p>}
  </form>
);
}

```

44. Form Libraries

For complex forms (many fields, cross-field validation, async submit, dynamic fields), use a form library. The top three are React Hook Form, Formik, and TanStack Form. React Hook Form is the most popular in 2024+ for its performance and Zod integration.

44.1 React Hook Form (Recommended)

```

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.string().email('Invalid email'),
  password: z.string().min(8, 'Min 8 chars'),
  remember: z.boolean().default(false)
});

type FormValues = z.infer<typeof schema>;

function LoginForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<FormValues>({ resolver: zodResolver(schema) });

  const onSubmit = async (data: FormValues) => {
    await fetch('/api/login', { method: 'POST', body: JSON.stringify(data) });
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>

```

```

<input {...register('email')} />
{errors.email && <span>{errors.email.message}</span>}

<input type="password" {...register('password')} />
{errors.password && <span>{errors.password.message}</span>}

<label>
  <input type="checkbox" {...register('remember')} /> Remember me
</label>

<button disabled={isSubmitting} type="submit">
  {isSubmitting ? 'Logging in...' : 'Login'}
</button>
</form>
);
}

```

44.2 Why React Hook Form?

- **Performance** — Uncontrolled by default (uses refs) — no re-render on every keystroke.
- **Tiny bundle** — ~9 KB gz; minimal runtime.
- **Schema validation** — Plug in Zod / Yup / Valibot via a resolver.
- **TypeScript-first** — Infer form types from the schema.
- **DevTools** — Inspect form state, values, errors visually.

45. Testing in React

Testing pyramid for React: unit tests (functions, hooks), component tests (React Testing Library), integration tests (multiple components together), end-to-end tests (Playwright, Cypress). The modern combination is Vitest + React Testing Library for unit/component, Playwright for E2E.

45.1 Test Types

Type	What	Tool	Notes
Unit	Pure functions, hooks	Vitest / Jest	Fast, isolated
Component	Single component in isolation	React Testing Library	Render, interact, assert
Integration	Multiple components /w routing, stores	RTL + MSW	Realistic interactions
E2E	Full app in real browser	Playwright, Cypress	Slow, flaky; tests user flows

45.2 Guiding Principles

- **Test behaviour, not implementation** — Test what the user sees and does, not internal state or methods. Refactor without breaking tests.
- **Query like a user** — Use `getByRole`, `getByLabelText`, `getByText`. Avoid `getByTestId` unless nothing else works.
- **Avoid implementation details** — Don't test state values, component internals, or private methods. They change during refactor.
- **MSW for API mocking** — Mock Service Worker intercepts network requests; the component sees real fetch behaviour without hitting the backend.

46. Jest

Jest is a JavaScript testing framework: test runner, assertions, mocks, snapshots. It was the default for Create React App. Modern alternatives: Vitest (Vite-native, faster, Jest-compatible API).

46.1 Basic Test

```
// math.test.ts
import { add, multiply } from './math';

describe('math', () => {
  test('adds two numbers', () => {
    expect(add(1, 2)).toBe(3);
  });

  test('multiplies two numbers', () => {
    expect(multiply(3, 4)).toBe(12);
  });

  test.each([[1, 1, 2], [2, 3, 5], [10, -5, 5]])(
    'add(%i, %i) = %i', (a, b, expected) => {
      expect(add(a, b)).toBe(expected);
    }
  );
});
```

46.2 Mocking

```
// Mock a module
jest.mock('./api', () => ({
  fetchUser: jest.fn().mockResolvedValue({ id: 1, name: 'Alice' })
}));

// Mock a function
const onClick = jest.fn();
render(<Button onClick={onClick} />);
fireEvent.click(screen.getByRole('button'));
expect(onClick).toHaveBeenCalledTimes(1);
expect(onClick).toHaveBeenCalledWith({ id: 1 });

// Spy on a method
const spy = jest.spyOn(console, 'log').mockImplementation();
doSomething();
expect(spy).toHaveBeenCalledWith('hello');
```

```
spy.mockRestore();
```

46.3 Vitest (Modern Alternative)

- **Vite-native** — Same config as your app (vite.config.ts), no separate Jest config.
- **Jest-compatible API** — describe, test, expect, vi.fn() — drop-in for most tests.
- **Faster** — ESM-native; no babel transform; runs in milliseconds.
- **Recommended for new Vite projects** — Jest still works but Vitest is the modern default.

47. React Testing Library

React Testing Library (RTL) is the standard for testing React components. It renders components into a real DOM (jsdom) and lets you query and interact with them the way a user would — by role, label, text — not by implementation detail.

47.1 Basic Component Test

```
// Login.test.tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Login from './Login';

test('submits email and password', async () => {
  const user = userEvent.setup();
  const onSubmit = jest.fn();
  render(<Login onSubmit={onSubmit} />);

  // Type into fields by label (accessibility-friendly query)
  await user.type(screen.getByLabelText(/email/i), 'alice@example.com');
  await user.type(screen.getByLabelText(/password/i), 'secret123');

  // Click submit
  await user.click(screen.getByRole('button', { name: /login/i }));

  // Assert onSubmit was called with the form data
  expect(onSubmit).toHaveBeenCalledWith({
    email: 'alice@example.com',
    password: 'secret123'
  });
});
```

47.2 Query Priority (Use in This Order)

Query	Use For
getByRole	Best — matches how assistive tech sees the page
getByLabelText	Forms — labels tie to inputs
getByPlaceholderText	Fallback if no label
getByText	Buttons, headings, paragraphs

Query	Use For
getByDisplayValue	Current value of form fields
getByAltText	Images and areas
getByTitle	Title attributes
getByTestId	Last resort — anything else first

48. TypeScript with React

TypeScript is the recommended language for React apps. It catches type errors at compile time, provides autocomplete in editors, and makes refactoring safer. Vite and Next.js both support TS out of the box.

48.1 Typing Props

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary' | 'outline'; // union
  disabled?: boolean;
  children?: React.ReactNode;
}

function Button({ label, onClick, variant = 'primary', disabled, children }:
ButtonProps) {
  return (
    <button
      className={`btn btn-${variant}`}
      onClick={onClick}
      disabled={disabled}
    >
      {label || children}
    </button>
  );
}
```

48.2 Typing Hooks

```
// useState with type argument
const [count, setCount] = useState<number>(0);
const [user, setUser] = useState<User | null>(null);

// useReducer
type Action = { type: 'increment' } | { type: 'set'; value: number };
function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'increment': return state + 1;
    case 'set': return action.value;
  }
}
const [state, dispatch] = useReducer(reducer, 0);

// useRef
const inputRef = useRef<HTMLInputElement>(null);
```

```
// useContext
const ThemeContext = createContext<'light' | 'dark'>('light');
const theme = useContext(ThemeContext); // typed as 'light' | 'dark'
```

48.3 Typing Events

```
function SearchBox() {
  const [q, setQ] = useState('');

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setQ(e.target.value);
  };

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    search(q);
  };

  const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log('Button clicked at', e.clientX, e.clientY);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={q} onChange={handleChange} />
      <button onClick={handleClick}>Search</button>
    </form>
  );
}
```

48.4 Utility Types

Type	Use
React.ComponentProps	Get props type of a component
React.ReactNode	Anything renderable (string, element, null)
React.ReactElement	A single JSX element
React.FC	Function component type (less used; verbose)
Pick	Subset of T's keys
Omit	T without specified keys
Partial	All keys optional
Required	All keys required

49. Folder Structure

There is no official "correct" folder structure, but a few patterns work well. The key principle: co-locate related files (component, test, styles, types) and group by feature, not by file type.

49.1 Feature-Based (Recommended)

```
src/
  features/
    auth/
      components/
        LoginForm.tsx
        SignupForm.tsx
      hooks/
        useAuth.ts
      api.ts
      types.ts
      AuthPage.tsx
    products/
      components/
        ProductList.tsx
        ProductCard.tsx
      hooks/
        useProducts.ts
      api.ts
      types.ts
    cart/
      ...
  components/ # Shared/generic UI components
    Button/
      Button.tsx
      Button.test.tsx
      Button.module.css
      index.ts
    Modal/
    Spinner/
  hooks/ # Shared hooks
    useFetch.ts
    useDebounce.ts
  lib/ # Third-party config (axios instance, dayjs)
  store/ # Global state (Redux/RTK slices)
  routes/ # Route definitions
  types/ # Shared types
  utils/ # Pure helpers
  App.tsx
  main.tsx # Entry point
```

49.2 Why Feature-Based?

- **Cohesion** — All files for a feature live together — easy to find, easy to delete.
- **Independent teams** — Team A works in features/auth; Team B in features/cart without conflicts.
- **Easy to extract** — A feature can be lifted into its own package or micro-frontend.
- **Avoids "components" folder with 200 files** — Grouping by type (all components together) does not scale.

50. Build Tools

A build tool transforms your source (TS, JSX, CSS Modules, tree-shaking, code splitting) into optimised production assets. The modern default is Vite; Create React App is deprecated; Next.js has its own build pipeline.

50.1 Vite (Recommended)

- **Dev server** — ESM-native — no bundling in dev. Starts in < 1 second regardless of app size.
- **HMR** — Hot Module Replacement — instant refresh on save.
- **Production build** — Rollup under the hood — efficient code splitting, tree shaking.
- **Plugins** — React, Vue, Svelte, MDX, PWA — wide ecosystem.
- **Created by Evan You** — Same author as Vue.
- **Adopted by** — React docs (new app scaffolding), Astro, SvelteKit, Nuxt 3.

50.2 Next.js

- **Full-stack framework** — Build, router, SSR/SSG, API routes, image optimization, deployment.
- **Built on React** — Same component model; adds App Router with server components.
- **Production features** — Edge runtime, ISR (incremental static regeneration), middleware.
- **Best for** — Marketing sites, e-commerce, anything SEO-sensitive, apps that benefit from server rendering.

51. Vite vs CRA

Create React App (CRA) was the official scaffolding tool for React SPAs from 2016 to 2023. It is now deprecated. Vite is the modern replacement — faster dev server, simpler config, smaller boilerplate.

51.1 Comparison

Aspect	CRA (webpack)	Vite
Dev server	Webpack dev server (bundles on every refresh)	ESM native (no bundling)
Cold start	~30 seconds for medium apps	~1 second
HMR speed	Seconds (rebuilds bundle)	Milliseconds (single module)
Production build	Webpack	Rollup
Config	Hidden, ejectable (painful)	vite.config.ts — simple
Maintenance	Deprecated Feb 2023	Active
Bundle size	Larger (more polyfills)	Smaller

51.2 Recommendation

- **New SPA** — Vite + React + TypeScript template.
- **New SSR app** — Next.js or Remix.
- **Existing CRA app** — Migrate to Vite (one-day effort for typical apps).
- **Enterprise webpack setups** — Stay on webpack if you have heavy custom config; otherwise migrate.

52. SSR vs CSR

CSR (Client-Side Rendering): the server sends an empty HTML shell; React renders in the browser. SSR (Server-Side Rendering): the server renders the initial HTML; React hydrates on the client. Each has trade-offs.

52.1 Comparison

Aspect	CSR	SSR
Initial load	Empty shell + JS, then renders	Pre-rendered HTML, ready to read
SEO	Poor (crawler sees empty shell)	Excellent (HTML has content)
First contentful paint	Slower (must run JS)	Faster
Time to interactive	Slower (must hydrate)	Similar after hydration
Server load	Low (static files)	High (renders per request)
Hosting	Static CDN (cheap)	Node server (more complex)
Best for	Apps (dashboards, admin tools)	Public sites (e-commerce, blogs)

52.2 Other Rendering Modes

- **SSG (Static Site Generation)** — Pre-render at build time. Best for blogs, docs, marketing sites.
- **ISR (Incremental Static Regeneration)** — SSG with periodic rebuilds (Next.js).
- **Streaming SSR** — Server streams HTML as it renders — faster first paint. Supported by React 18 and Next.js 13+.
- **React Server Components** — Components that render only on the server, no JS shipped. Next.js App Router.

53. Next.js Basics

Next.js is the most popular React framework. Built on React, it adds: file-based routing (App Router), server components, SSR/SSG/ISR, API routes, image/font optimization, and built-in deployment to Vercel. The current version is Next.js 15 with the App Router (introduced in 13).

53.1 App Router Structure

```
app/
  layout.tsx # Root layout (HTML, body, providers)
  page.tsx # Home page (/)
  globals.css
  about/
    page.tsx # /about
  blog/
    page.tsx # /blog (list)
    [slug]/
      page.tsx # /blog/:slug (dynamic)
    layout.tsx # Blog section layout
  api/
    users/
      route.ts # /api/users (server-side handler)
  components/ # Component co-located with route
```

53.2 Server vs Client Components

```
// app/page.tsx – Server Component by default (no JS shipped)
import { db } from '@lib/db';

export default async function Page() {
  const products = await db.product.findMany(); // direct DB access
  return (
    <ul>
      {products.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}

// 'use client' directive marks a Client Component
'use client';
import { useState } from 'react';

export function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

53.3 Data Fetching

```
// Server Component: fetch with caching/revalidation
export default async function Products() {
  const res = await fetch('https://api.example.com/products', {
    next: { revalidate: 60 } // ISR – revalidate every 60s
  });
  const products = await res.json();
  return <ProductList products={products} />;
}
```



```
// Dynamic route
export default async function ProductPage({ params }: { params: { slug: string } }) {
  const res = await fetch(`https://api.example.com/products/${params.slug}`);
  const product = await res.json();
  return <ProductDetail product={product} />;
}
```

54. Virtual DOM

The Virtual DOM (VDOM) is a lightweight JavaScript representation of the real DOM. React keeps two VDOM trees: the current one and the next one. On every state change, React diffs them and applies only the net changes to the real DOM — far cheaper than directly manipulating the DOM.

54.1 Why VDOM?

- **DOM manipulation is slow** — Each layout/style recalculation (reflow) costs milliseconds. Direct DOM updates from JS cause many small refloWS.
- **Batched updates** — React collects state changes and applies one batched update — single reflow for many state changes.
- **Declarative code** — You describe the desired UI; React figures out the minimal diff to apply.
- **Cross-platform** — The VDOM is just JS objects — same engine powers React DOM, React Native, React Three Fiber.

54.2 VDOM Tree Example

```
// JSX:
<div className="card">
  <h1>Hello</h1>
  <p>World</p>
</div>

// Compiles to React elements (the VDOM):
{
  type: 'div',
  props: {
    className: 'card',
    children: [
      { type: 'h1', props: { children: 'Hello' } },
      { type: 'p', props: { children: 'World' } }
    ]
  }
}
```

55. Reconciliation & Diffing

Reconciliation is the process React uses to update the real DOM to match the latest VDOM. It uses a diffing algorithm to compare the previous and new VDOM trees and produce the minimum set of DOM mutations.

55.1 Diffing Heuristics

- **Different types → different trees** — If a node changes from `<div>` to `<span>`, or from `<Counter />` to `<Toggle />`, React tears down the old subtree and builds a new one. No reuse.
- **Same type → preserve DOM node** — React updates attributes (`className`, `style`) and recurses into children.
- **List children keyed by `key`** — React matches list items across renders by key. Stable keys = reuse DOM nodes; unstable keys = rebuild.
- **O(n) algorithm** — Tree diffing is $O(n^3)$ in general; React's heuristics make it $O(n)$ by assuming "different types = different trees".

55.2 Keys in Reconciliation

```
// Before: [A, B, C]
<ul>
  <li key="a">A</li>
  <li key="b">B</li>
  <li key="c">C</li>
</ul>

// After: [Z, A, B, C] – Z inserted at front
<ul>
  <li key="z">Z</li>
  <li key="a">A</li>
  <li key="b">B</li>
  <li key="c">C</li>
</ul>

// With keys: React adds Z at front, keeps A/B/C in place – 1 DOM op.
// Without keys (or with index keys): React reuses the first 3 DOM
// nodes, sets their text to Z/A/B, and adds a new one for C – 4 ops.
```

56. React Fiber

Fiber is React's reimplementation of the reconciliation engine, introduced in React 16 (2017). It enables incremental rendering — React can pause, resume, or abort rendering work, allowing concurrent features (time slicing, transitions, Suspense) in React 18+.

56.1 Why Fiber?

- **Before Fiber** — React's stack-based reconciler processed the entire tree synchronously — could not be interrupted. Long renders blocked the main thread, causing jank.
- **Fiber** — Each component is a "fiber" — a unit of work. React can pause, resume, or throw away work. Priority-based: urgent updates (user input) pre-empt non-urgent (data fetching).
- **Concurrent features** — `useTransition`, `useDeferredValue`, `Suspense`, streaming SSR — all rely on Fiber's interruptible rendering.

56.2 Fiber Tree

- **Each fiber** — JS object representing a component instance + its work state.
- **Linked-list structure** — child, sibling, return pointers — lets React walk the tree without recursion (no stack overflow).
- **Two phases** — Render phase (pure, can be interrupted) and Commit phase (applies DOM changes, runs effects — synchronous).

58. Machine Coding Questions

Machine coding rounds ask you to build a small working component in 45-60 minutes. The expectation: clean code, working state, basic styling, edge cases handled. Below are common patterns.

58.1 Build a Counter

```
function Counter({ initial = 0, step = 1, min = -Infinity, max = Infinity }) {
  const [count, setCount] = useState(initial);

  const inc = () => setCount(c => Math.min(max, c + step));
  const dec = () => setCount(c => Math.max(min, c - step));
  const reset = () => setCount(initial);

  return (
    <div>
      <button onClick={dec} disabled={count <= min}>-</button>
      <span>{count}</span>
      <button onClick={inc} disabled={count >= max}>+</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

58.2 Build a Todo List

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');
  const [filter, setFilter] = useState('all');

  const addTodo = () => {
    if (!input.trim()) return;
    setTodos(prev => [...prev, { id: Date.now(), text: input, done: false }]);
    setInput('');
  };

  const toggle = (id) =>
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t));

  const remove = (id) => setTodos(prev => prev.filter(t => t.id !== id));

  const visible = todos.filter(t =>
    filter === 'all' ? true : filter === 'active' ? !t.done : t.done
  );

  return (
    <div>
```

```


<button onClick={addTodo}>Add</button>

<div>
  {[ 'all', 'active', 'done' ].map(f => (
    <button key={f} onClick={() => setFilter(f)}
      style={{ fontWeight: filter === f ? 'bold' : 'normal' }}>
        {f}
      </button>
    )
  )}
</div>

<ul>
  {visible.map(t => (
    <li key={t.id}>
      <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
      <span style={{ textDecoration: t.done ? 'line-through' : 'none' }}>
        {t.text}
      </span>
      <button onClick={() => remove(t.id)}>x</button>
    </li>
  )
  )}
</ul>
</div>
);
}

```

58.3 Build an Autocomplete / Typeahead

```

function Autocomplete({ fetchSuggestions, onSelect }) {
  const [query, setQuery] = useState('');
  const [suggestions, setSuggestions] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [open, setOpen] = useState(false);
  const [highlight, setHighlight] = useState(0);

  // Debounce the query
  const debounced = useDebounce(query, 300);

  useEffect(() => {
    if (!debounced) { setSuggestions([]); return; }
    let cancelled = false;
    setIsLoading(true);
    fetchSuggestions(debounced)
      .then(items => { if (!cancelled) { setSuggestions(items); setOpen(true); } })
      .finally(() => { if (!cancelled) setIsLoading(false); });
    return () => { cancelled = true; };
  }, [debounced]);

  const handleKey = (e) => {
    if (e.key === 'ArrowDown') setHighlight(h => Math.min(h + 1, suggestions.length - 1));
    if (e.key === 'ArrowUp') setHighlight(h => Math.max(h - 1, 0));
    if (e.key === 'Enter' && suggestions[highlight]) {
      onSelect(suggestions[highlight]);
      setOpen(false);
    }
  }
}

```

```

    if (e.key === 'Escape') setOpen(false);
  };

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        onKeyDown={handleKey}
        placeholder="Search..."
      />
      {isLoading && <span>...</span>}
      {open && suggestions.length > 0 && (
        <ul>
          {suggestions.map((s, i) => (
            <li key={s.id}
              style={{ background: i === highlight ? '#eee' : 'white' }}
              onMouseEnter={() => setHighlight(i)}
              onClick={() => { onSelect(s); setOpen(false); }}>
              {s.label}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

```

58.4 Common Machine Coding Tasks

- **Counter with min/max/step** — Tests useState, controlled inputs, edge cases.
- **Todo list with filter** — Tests list rendering, keys, controlled inputs, conditional rendering.
- **Autocomplete / typeahead** — Tests async, debounce, keyboard navigation, accessibility.
- **Modal / dialog** — Tests portals, focus trap, ESC to close, click outside.
- **Accordion** — Tests state for active panel, conditional rendering.
- **Star rating** — Tests hover state, click to set.
- **Tic-tac-toe** — Tests state, derived values, win detection.
- **Pagination component** — Tests props, derived pages, event handlers.
- **Image carousel** — Tests effects (auto-advance), state, transitions.
- **Infinite scroll list** — Tests IntersectionObserver, paginated fetching, loading state.

59. Coding Challenges

Coding rounds typically ask JS fundamentals (debounce, throttle, promise.all) or React-specific patterns (custom hooks). Below are common patterns with model answers.

59.1 Implement Debounce

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

// As a React hook
function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

59.2 Implement Throttle

```
function throttle(fn, limit) {
  let inThrottle = false;
  return function (...args) {
    if (inThrottle) return;
    fn.apply(this, args);
    inThrottle = true;
    setTimeout(() => { inThrottle = false; }, limit);
  };
}
```

59.3 Implement Promise.all

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = new Array(promises.length);
    let completed = 0;
    if (promises.length === 0) return resolve([]);
    promises.forEach((p, i) => {
      Promise.resolve(p)
        .then(value => {
          results[i] = value;
          if (++completed === promises.length) resolve(results);
        })
        .catch(reject);
    });
  });
}
```

59.4 Implement usePrevious Hook

```
function usePrevious(value) {
  const ref = useRef(value);
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current;
}
```

```
// Usage: get the previous value of a prop or state
function Counter() {
  const [count, setCount] = useState(0);
  const prev = usePrevious(count);
  return <p>Now: {count}, was: {prev}</p>;
}
```

59.5 Implement setInterval Hook

```
// setInterval that respects prop changes (e.g., dynamic delay)
function useInterval(callback, delay) {
  const savedCallback = useRef(callback);

  useEffect(() => {
    savedCallback.current = callback;
  }, [callback]);

  useEffect(() => {
    if (delay === null) return;
    const id = setInterval(() => savedCallback.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}
```

60. System Design for Frontend

Frontend system design rounds test whether you can architect a large-scale UI: component hierarchy, state management, data flow, performance, accessibility, and trade-offs. Common questions: design Netflix, design Twitter feed, design a chat app, design an autocomplete.

60.1 Six-Step Framework

Step	What to Do
1. Clarify	Functional + non-functional requirements; scale, performance, devices
2. Architecture	High-level diagram: client, CDN, API, state layers
3. Component breakdown	Top-down decomposition: pages → components
4. State management	Local vs global vs server state; choose tools
5. Data flow	API contract, caching, optimistic updates
6. Optimize	Lazy load, virtualize, accessibility, error handling

60.2 Example: Design an Autocomplete Search

- **Requirements** — Type → see suggestions; debounced; keyboard nav; cache recent queries; min 3 chars; show top 10; handle empty/error states.
- **API** — GET /api/suggest?q=foo&limit;=10 → string[].

- **Components** — Autocomplete wrapper, Input, SuggestionList, SuggestionItem, LoadingSpinner, EmptyState.
- **State** — Local: query, suggestions, isLoading, highlight, open. Cached: recent queries in localStorage.
- **Debounce** — 300ms — avoid request per keystroke.
- **Caching** — TanStack Query with queryKey: ['suggest', q]. Same query in 100ms → 1 request.
- **Edge cases** — Empty input, no results, network error, slow network, abort in-flight on new keystroke (AbortController).
- **Accessibility** — ARIA combobox role, aria-expanded, aria-activedescendant for highlight, keyboard arrows + Enter + Escape.

60.3 Performance Checklist

- **Code splitting** — Lazy-load routes and heavy features.
- **Bundle size** — Audit with webpack-bundle-analyzer; tree-shake; avoid large libs.
- **Caching** — TanStack Query for server state; service worker for offline.
- **Virtualization** — react-window for lists > 100 items.
- **Image optimization** — next/image or lazy-load + responsive sizes.
- **Prefetch** — Prefetch next route on hover/focus.
- **Debounce** — Search, resize, scroll handlers.
- **Critical CSS** — Inline above-the-fold CSS for fast first paint.

61. Best Practices

A consolidated checklist of React best practices. Following these prevents most production issues and keeps code maintainable.

61.1 Component Design

- **Single responsibility** — One component does one thing. If it has 3+ unrelated concerns, split.
- **PascalCase names** — Button, ProductList — distinguishes from HTML tags.
- **Props are read-only** — Never mutate props. Use state for changes.
- **Default props** — Provide sensible defaults via destructuring.
- **Composition over inheritance** — Use children, render props, hooks.
- **Co-locate related files** — Component, test, styles, types together.

61.2 State

- **Lift state up, not across** — For sibling-shared state, lift to common parent.
- **Use the right tool** — Local for component; Context for global rarely-changing; Redux/Zustand for complex; TanStack Query for server.
- **Derive, don't store** — Compute derived values during render; do not store them in state.
- **Immutable updates** — Always create new objects/arrays. Use Immer for nested updates.

- **Lazy init** — `useState()` => `expensiveInit()` for costly initial values.

61.3 Effects

- **Use the dependency array** — List every value used; enable `exhaustive-deps` lint.
- **Cleanup everything** — Listeners, timers, subscriptions, `AbortController` for fetch.
- **Don't store derived state** — Use `useMemo` or compute during render.
- **Prefer TanStack Query** — For server state, skip `useEffect` + `fetch`.

61.4 Performance

- **Profile before optimising** — React DevTools Profiler identifies real bottlenecks.
- **Memoise selectively** — `React.memo` for pure components in re-render-heavy trees; `useMemo` for expensive computations; `useCallback` for stable callbacks.
- **Code split** — `React.lazy` per route.
- **Virtualise long lists** — `react-window` for > 100 items.
- **Stable keys** — Use stable unique IDs from data, never array index for reorderable lists.

61.5 Accessibility

- **Semantic HTML** — `<button>` not `<div onClick>`; `<a>` for navigation.
- **Labels** — Every input has a label; use `aria-label` only when no visible label fits.
- **Keyboard nav** — Tab order, focus styles, Enter/Escape handlers.
- **ARIA** — Use sparingly; only when semantic HTML cannot express the pattern.
- **Color contrast** — WCAG AA minimum (4.5:1 for body text).
- **Test with screen reader** — VoiceOver (Mac), NVDA (Windows).

61.6 Testing

- **Test behaviour** — What the user sees/does, not implementation.
- **Query by role/label** — `getByRole`, `getByLabelText` — accessible queries.
- **MSW for API mocking** — Realistic; tests the same fetch path as production.
- **Test critical paths** — Login, checkout, key flows. Skip trivial components.

62. React Project Ideas

Building real projects is the best way to learn. Below are 12 ideas ranging from beginner to advanced, each teaching specific concepts.

62.1 Beginner (1-2 days each)

Project	Concepts
Todo List	useState, list rendering, controlled inputs, filter
Weather App	API calls (useEffect + fetch), conditional rendering
Calculator	useState, event handling, edge cases
Counter with history	useState arrays, undo/redo patterns
Markdown previewer	Forms, third-party lib (marked), live update

62.2 Intermediate (1-2 weeks each)

Project	Concepts
Movie DB search	TanStack Query, routing, pagination, search debounce
E-commerce cart	Context + useReducer, localStorage, optimistic UI
Blog with MDX	Routing, dynamic params, MDX, SSG
Real-time chat	WebSocket, useReducer, scroll handling, presence
Kanban board	Drag-and-drop (dnd-kit), state management, persistence

62.3 Advanced (1-2 months)

Project	Concepts
Full-stack app with Next.js	App Router, server components, Prisma, auth
Spotify-like player	Audio API, complex state, drag-drop playlists
Collaborative doc editor	WebSockets, OT/CRDT, optimistic updates
Admin dashboard	Charts, virtualized tables, RBAC, dark mode
Reddit clone	Voting, nested comments, infinite scroll, SSR

62.4 Tips for Portfolio Projects

- **Deploy it** — Vercel / Netlify — recruiters will not run your code locally.
- **Write a README** — What it does, tech stack, screenshots, live link, what you learned.
- **Add tests** — Even a few — shows you know how.
- **Use TypeScript** — All new projects; it is the industry standard.
- **Pick 2-3 deep projects** — Better than 10 shallow ones. Show range: one CRUD, one real-time, one with heavy state.

63. HR Interview Questions

HR and behavioural rounds test communication, conflict resolution, ownership, and cultural fit. Use the STAR framework (Situation, Task, Action, Result) for every behavioural answer. Prepare 3-5 concrete stories from your React work.

63.1 STAR Framework

Letter	Cover	Example
Situation	Set context in 1-2 sentences	At Acme, our React app's bundle was 2MB
Task	Your responsibility	I was the frontend lead
Action	What YOU did (not "we")	I introduced code splitting, lazy-loaded routes, audited bundle
Result	Quantified outcome	Bundle dropped to 350KB; LCP improved from 4s to 1.2s

63.4 Questions to Ask the Interviewer

- What does the team's React stack look like — Vite, Next.js, Remix? TypeScript?
- How does the team handle state — Context, Redux Toolkit, Zustand?
- What is the testing strategy — unit, integration, E2E?
- How does the team handle deployments — CI/CD, preview environments, feature flags?
- What is the biggest performance or scalability challenge the team is facing?
- What does success look like for this role in the first 6 months?
- How does the team balance feature work with refactoring and tech debt?
- What is the team's stance on adopting React 19 / server components?

63.5 Final Tips

- Think aloud during coding rounds; silent candidates look stuck.
- Ask clarifying questions before writing code — requirements, edge cases, expected output.
- Start with the simplest correct solution, then optimise — do not jump to fancy tricks.
- For machine coding, prioritise working code over perfect code. Show you can deliver.
- Quantify impact in behavioural answers (60% bundle reduction, 1.2s LCP, 50K users).
- Have 3-5 STAR stories ready: a perf win, a hard bug, a design decision, a conflict, a leadership moment.
- Know your resume cold — anything on it is fair game.
- Sleep well the night before. A rested brain beats last-minute cramming.

Final reminder: 0-3 years of experience means you should be able to discuss trade-offs, not just recite definitions. Whenever an interviewer asks "what is X?", answer with what it is, when to use it, and one limitation. The third sentence is what separates junior from mid-level candidates.